

Estrutura de Dados e Algoritmos

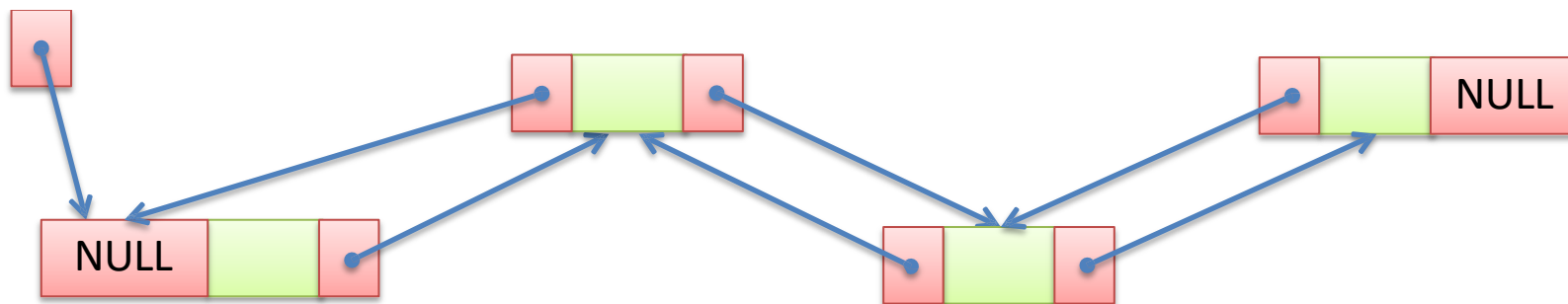
Listas Duplamente Encadeadas

- Na aula passada, vimos listas encadeadas simples, em que cada nó aponta apenas para o próximo. Hoje vamos melhorar essa estrutura adicionando um segundo ponteiro em cada nó: o ponteiro para o anterior. Isso nos permitirá percorrer a lista em dois sentidos e realizar algumas operações com mais flexibilidade.

- Uma lista duplamente encadeada é uma estrutura composta por nós, onde cada nó tem uma informação e dois ponteiros: um para o próximo elemento da lista, outro para o elemento anterior;
- Conhecido o primeiro elemento da lista (cabeça) ou o último (calda), é possível acessar os demais elementos;

- Diferente das listas encadeadas, as listas duplamente encadeadas reduzem a complexidade da remoção de um elemento do final da lista de $O(n)$ para $O(1)$;
- Os nós de uma lista duplamente encadeada apontam tanto para o sucessor (campo prox) quanto para o antecessor (campo ant) do elemento;
- Na lista simplesmente encadeada, cada nó sabe apenas quem vem depois. Na lista duplamente encadeada, cada nó sabe quem vem depois e quem vem antes;
- A lista dupla permite percorrer a estrutura nos dois sentidos e facilita algumas remoções, especialmente quando já temos referência ao nó ou ao último elemento.
- A lista duplamente encadeada aumenta o controle sobre a estrutura, mas também aumenta a responsabilidade: agora, em cada inserção ou remoção, precisamos manter dois sentidos de ligação consistentes.

Lista



```
struct TNo
```

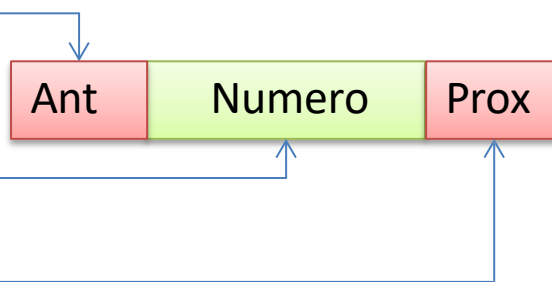
```
{
```

```
    TNo *Ant;
```

```
    int Numero;
```

```
    TNo *Prox;
```

```
};
```



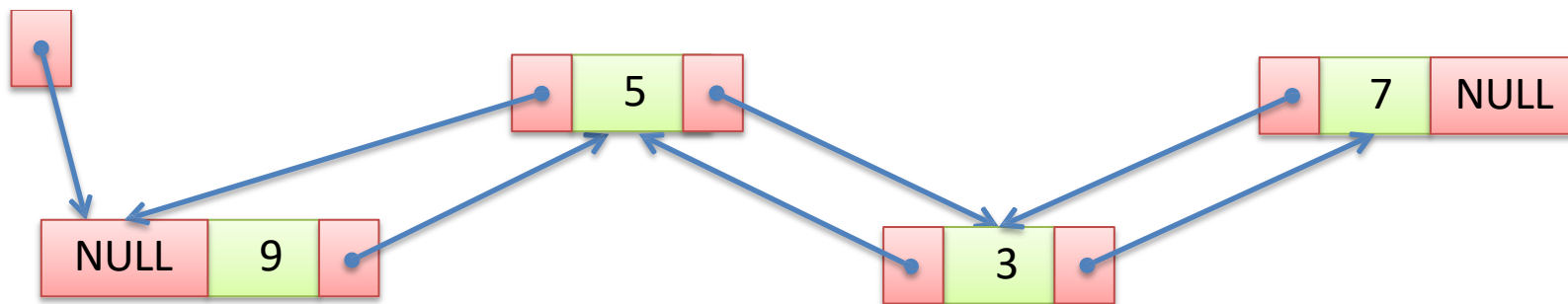
```
bool Vazia(TNo *pLista)
{
    if(pLista == NULL)
        return true;
    else
        return false;
}
```

Lista

NULL

Em uma lista duplamente encadeada, cada nó conhece quem vem antes e quem vem depois. Isso permite percorrer a lista nos dois sentidos, mas exige mais cuidado ao inserir e remover elementos, porque precisamos atualizar dois ponteiros: Ant e Prox.

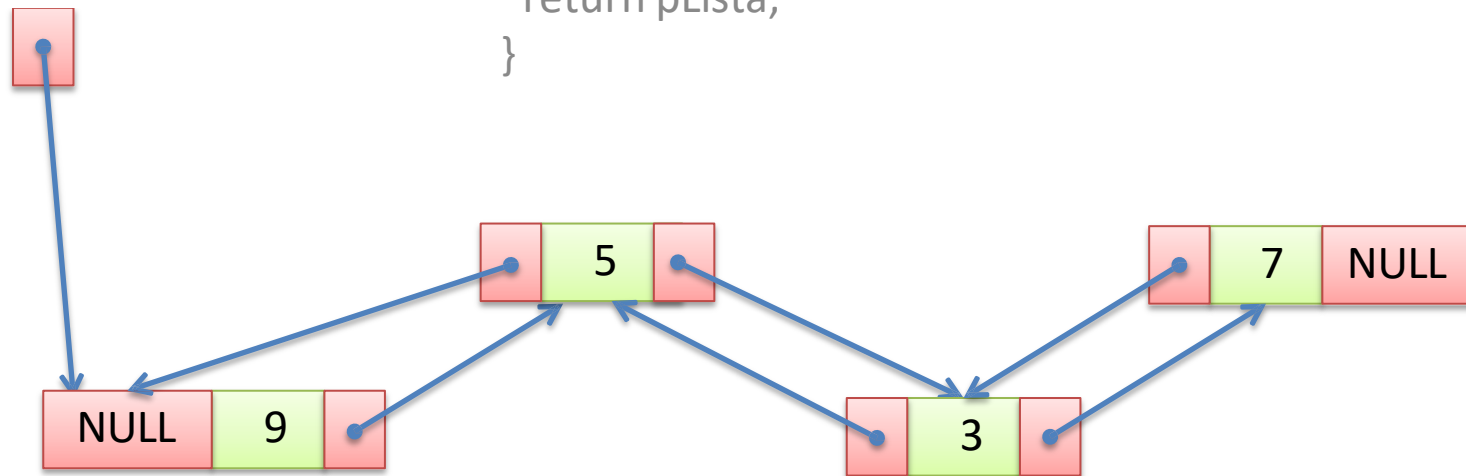
Lista



Lista Duplamente Encadeada Inclusão na Cabeça

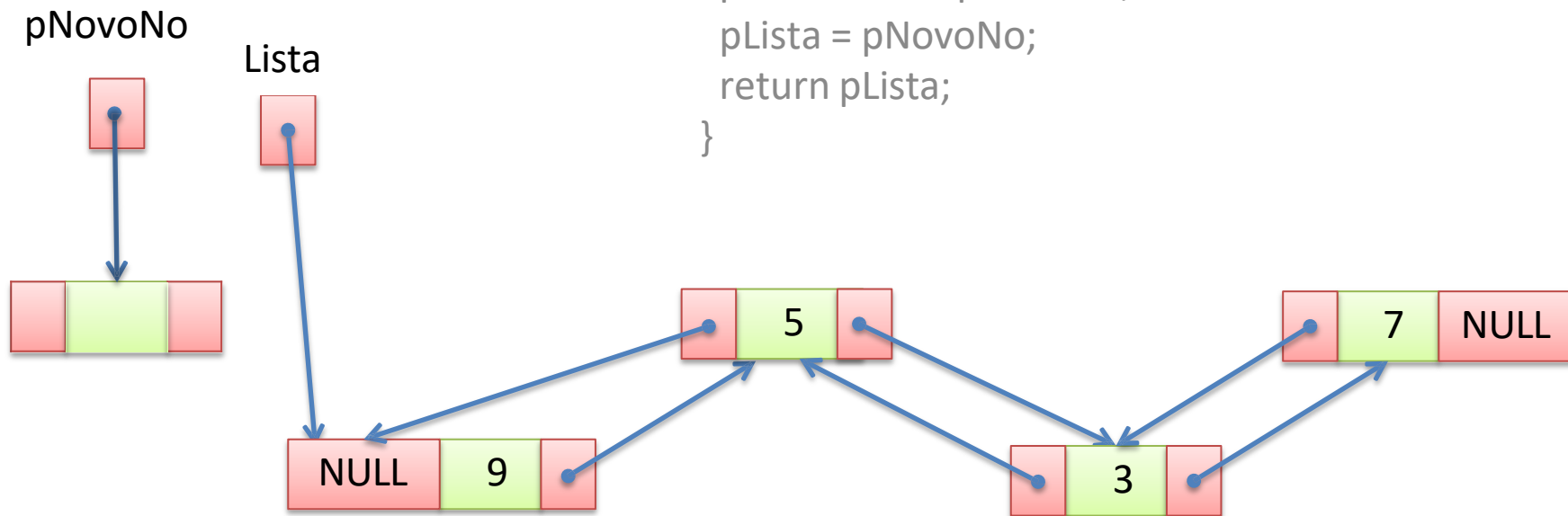
```
TNo *IncluiCabeça(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```

Lista



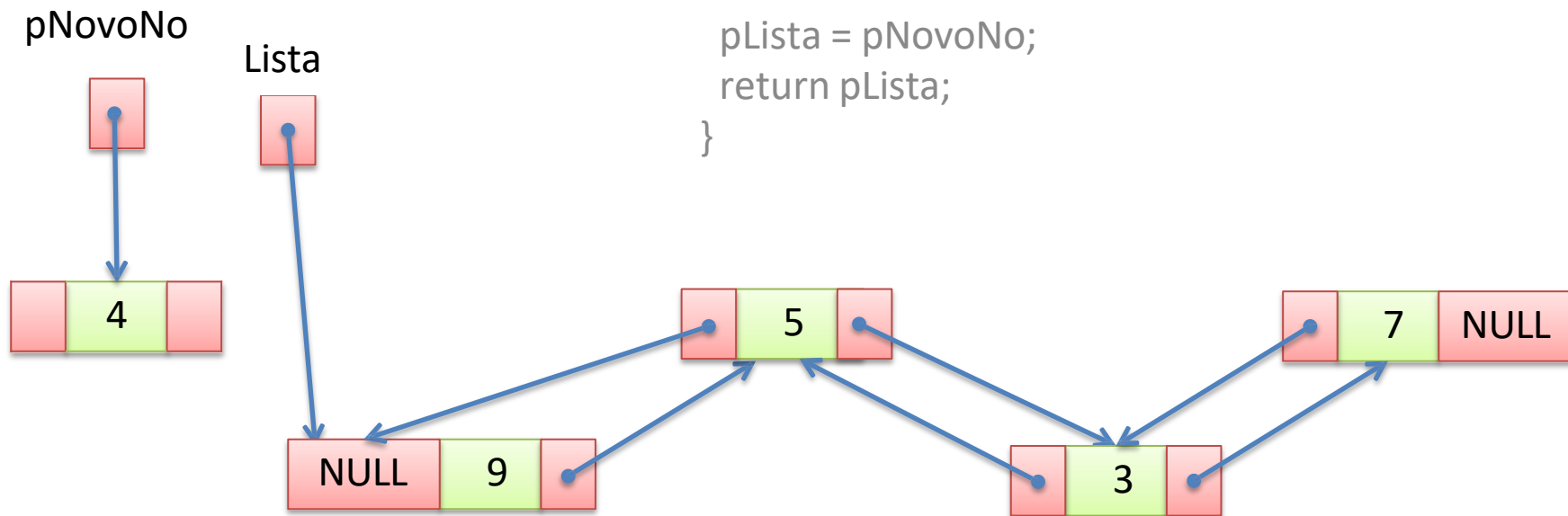
Lista Duplamente Encadeada Inclusão na Cabeça

```
TNo *IncluiCabeça(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```



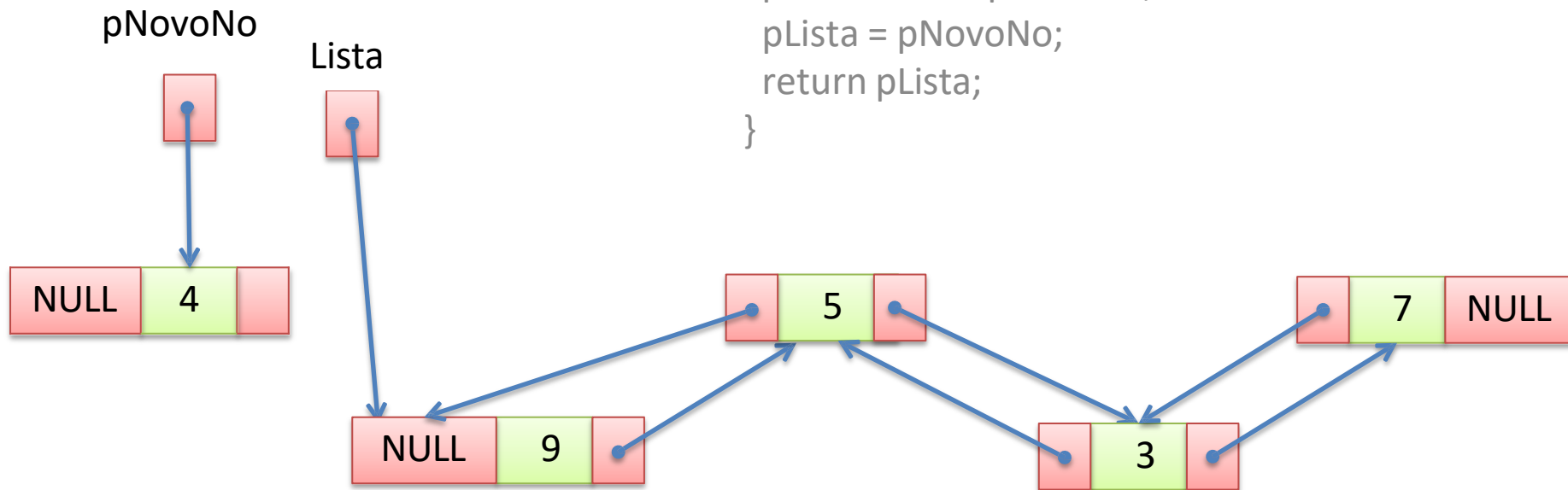
Lista Duplamente Encadeada Inclusão na Cabeça

```
TNo *IncluiCabeca(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```



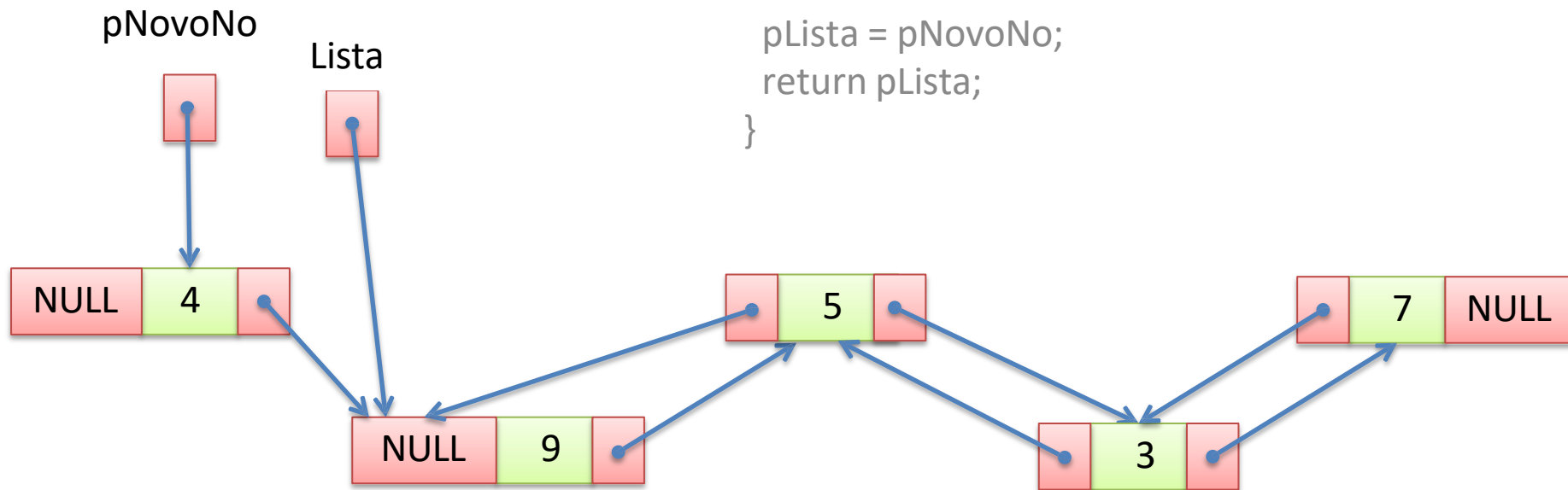
Lista Duplamente Encadeada Inclusão na Cabeça

```
TNo *IncluiCabeca(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```



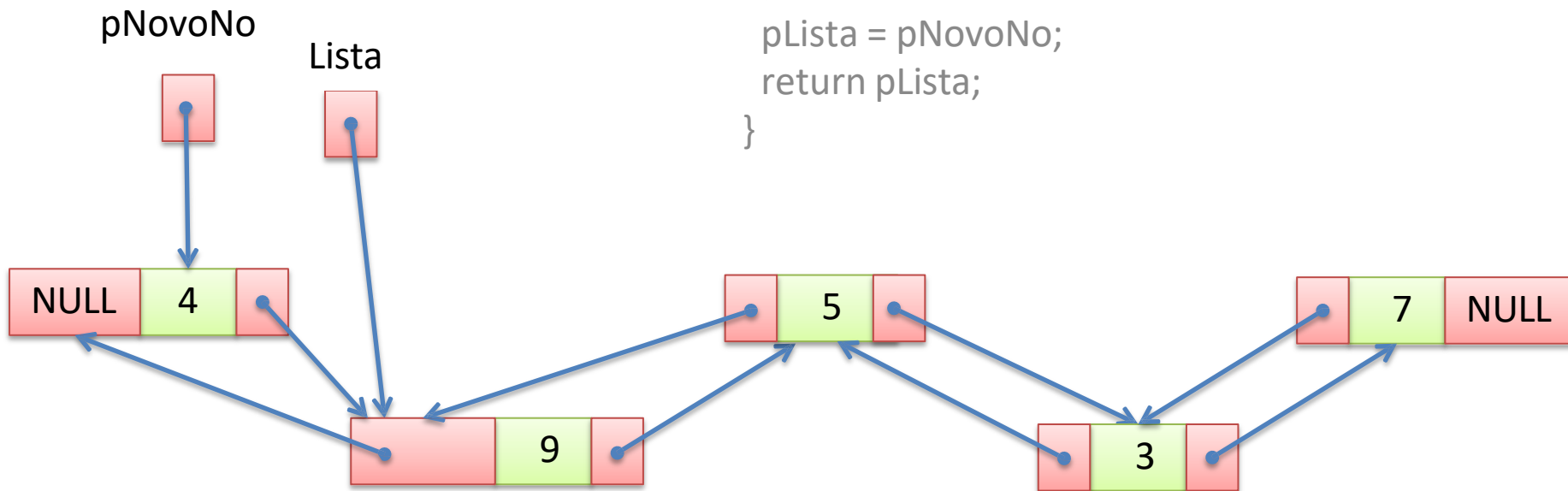
Lista Duplamente Encadeada Inclusão na Cabeça

```
TNo *IncluiCabeça(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```



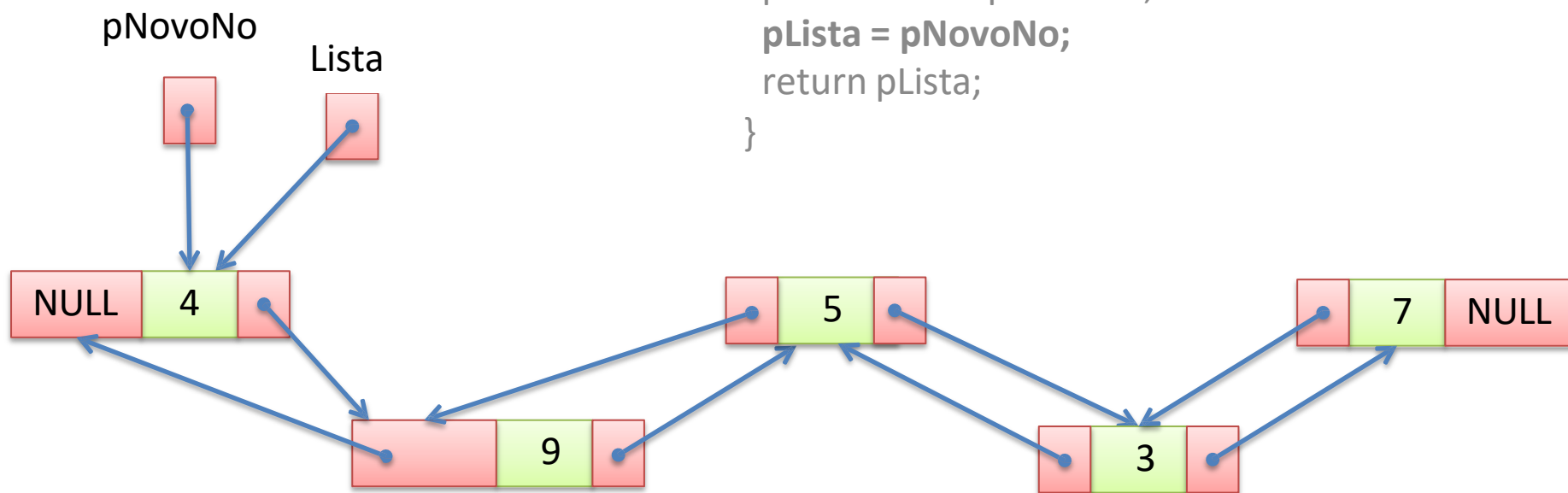
Lista Duplamente Encadeada Inclusão na Cabeça

```
TNo *IncluiCabeça(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```



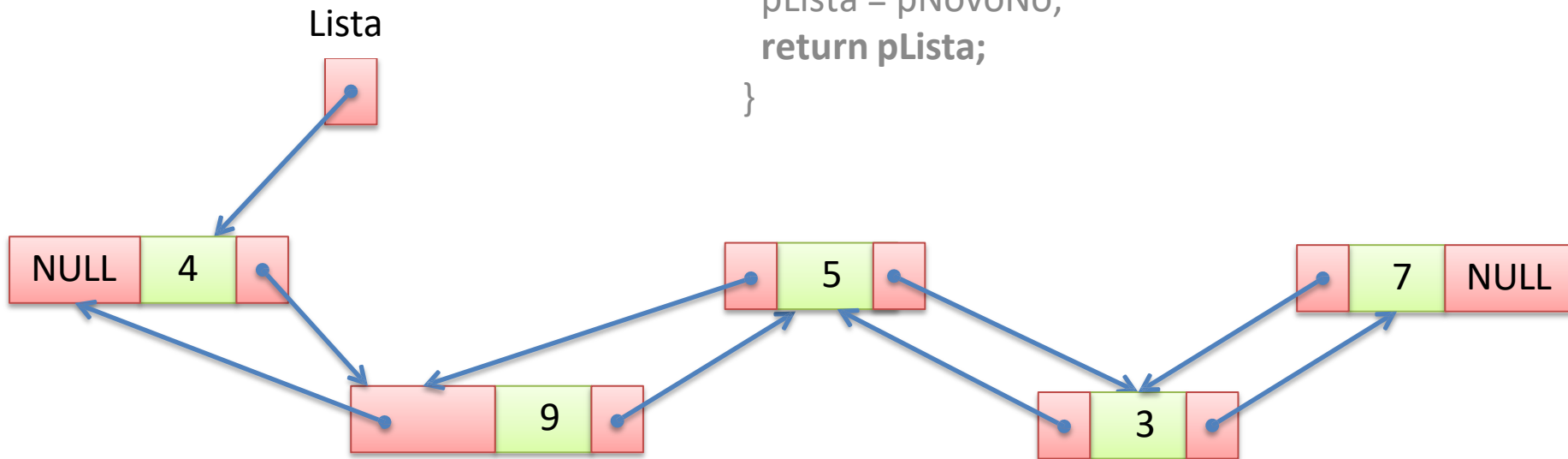
Lista Duplamente Encadeada Inclusão na Cabeça

```
TNo *IncluiCabeca(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```



Lista Duplamente Encadeada Inclusão na Cabeça

```
TNo *IncluiCabeça(TNo *pLista, int pValor)
{
    TNo *pNovoNo;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Ant = NULL;
    pNovoNo->Prox = pLista;
    pLista->Ant = pNovoNo;
    pLista = pNovoNo;
    return pLista;
}
```



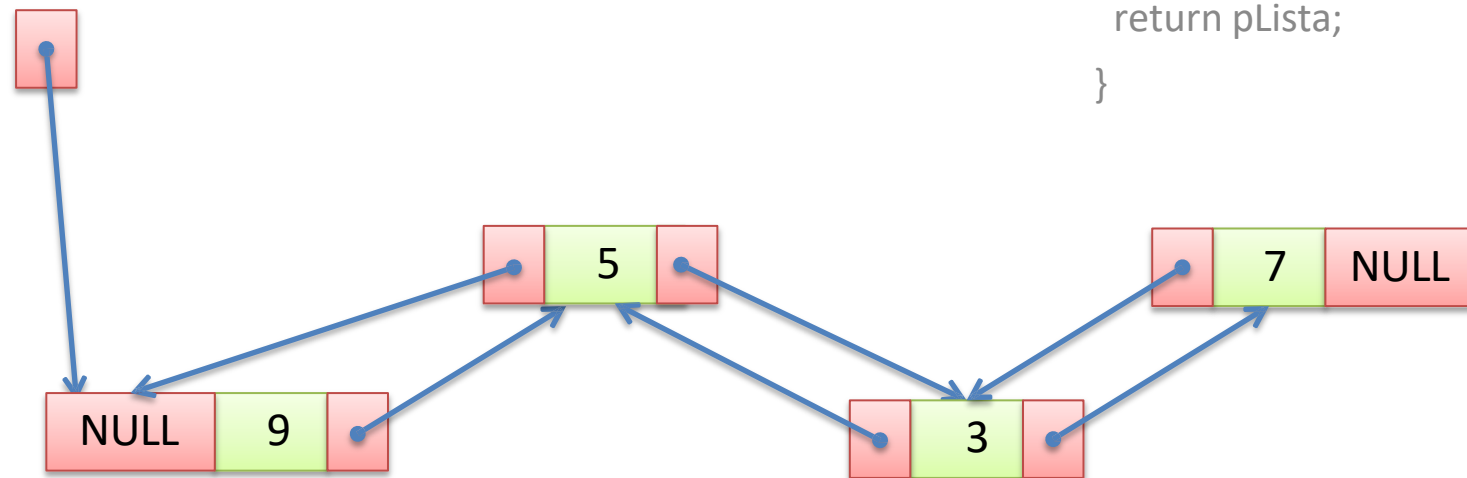
Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

Lista

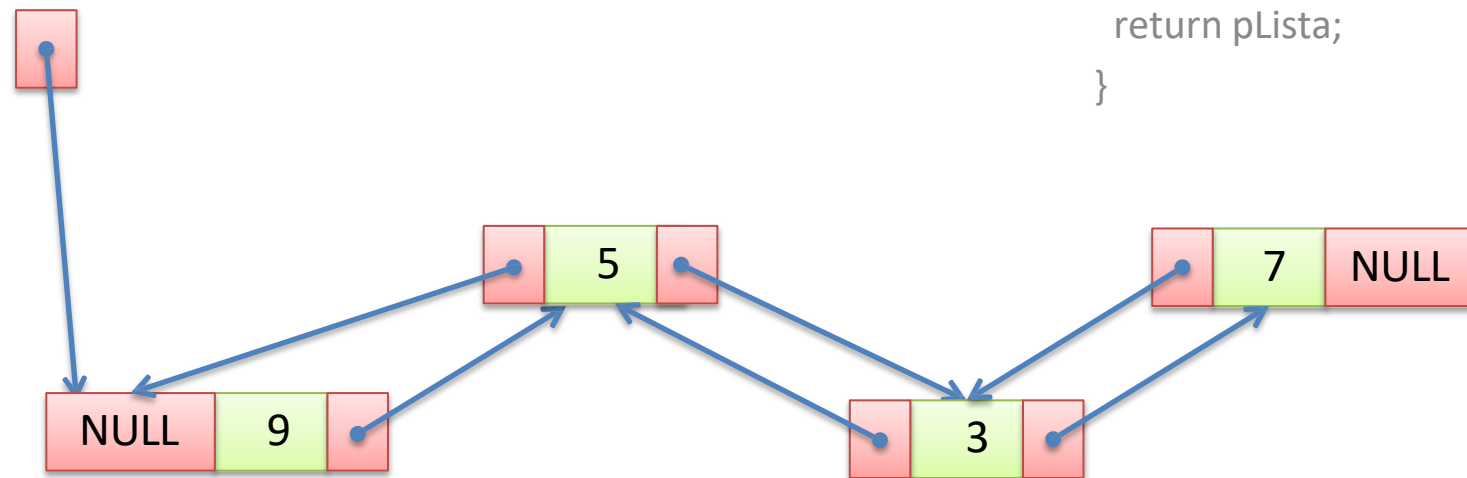


Lista Duplamente Encadeada

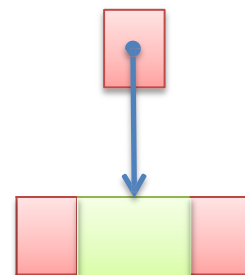
Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

Lista



pNovoNo



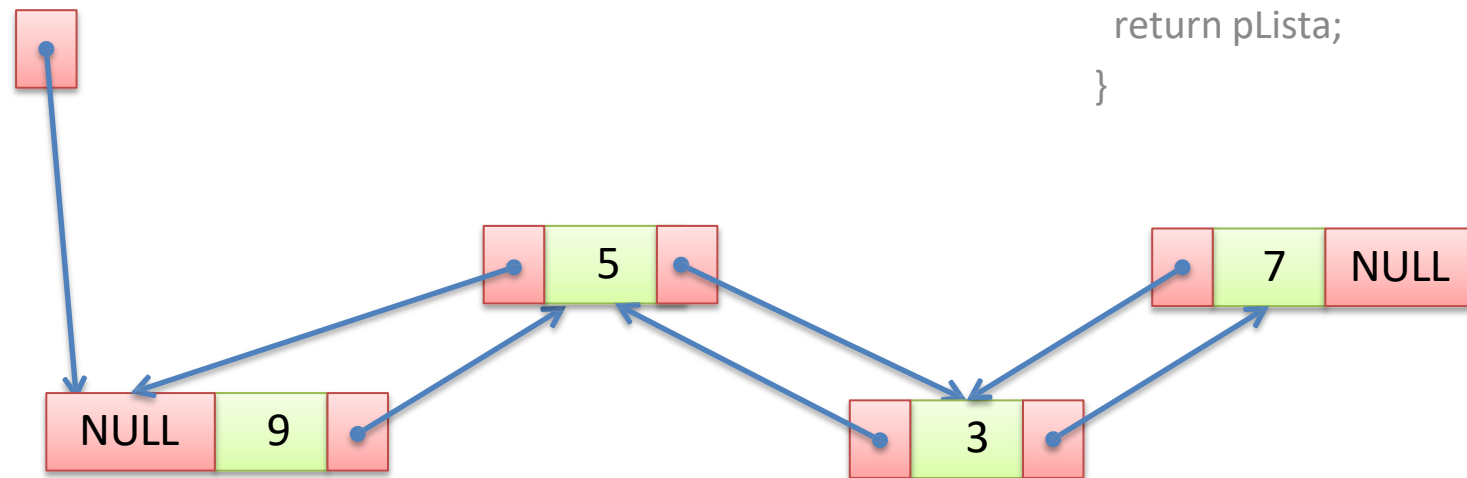
Lista Duplamente Encadeada

Inclusão na Calda

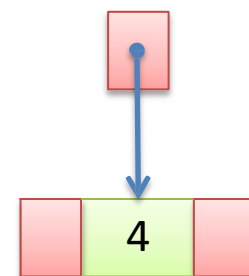
```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

Lista



pNovoNo



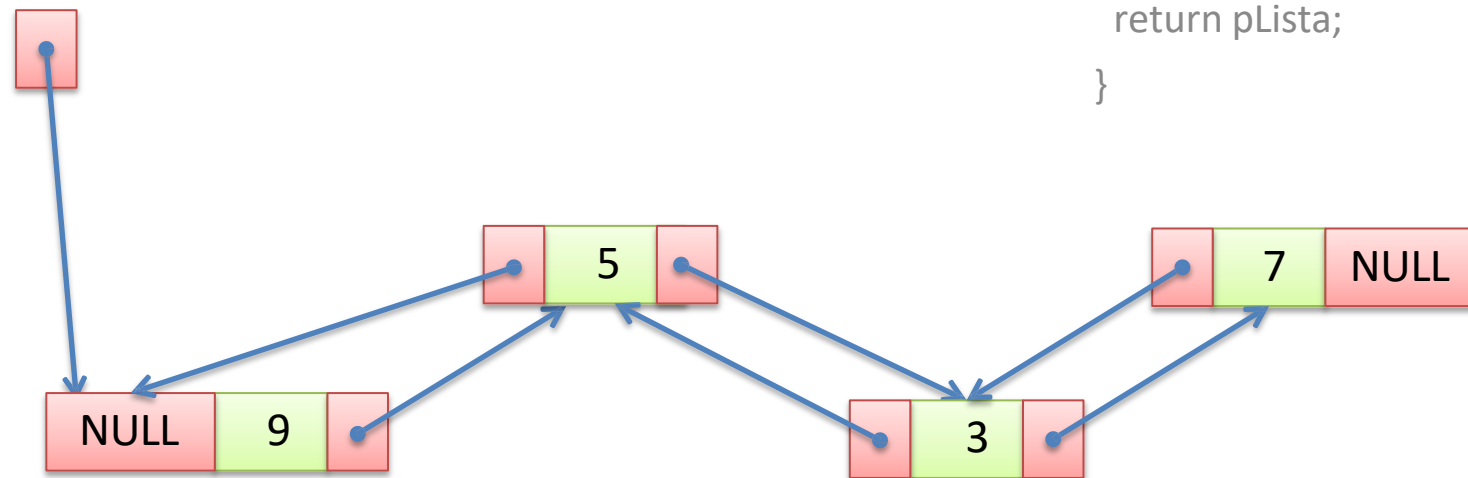
Lista Duplamente Encadeada

Inclusão na Calda

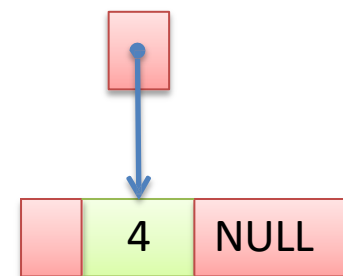
```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

Lista



pNovoNo



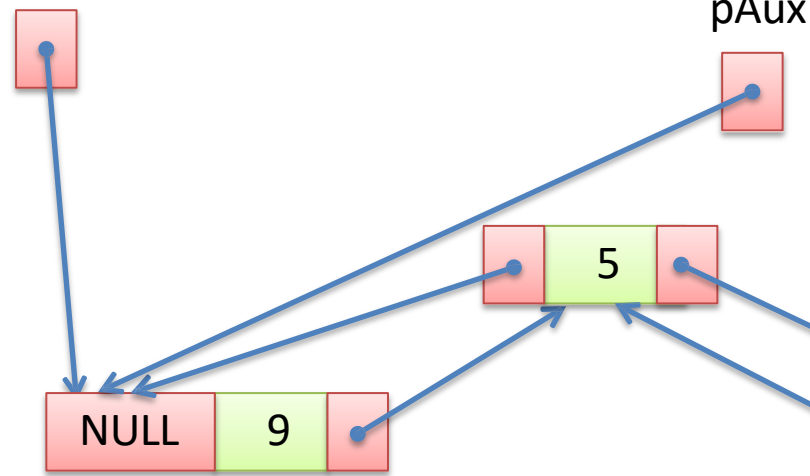
Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

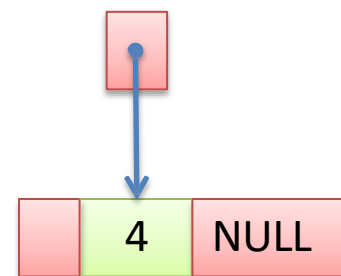
Lista



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

pNovoNo



Estrutura de Dados e Algoritmos

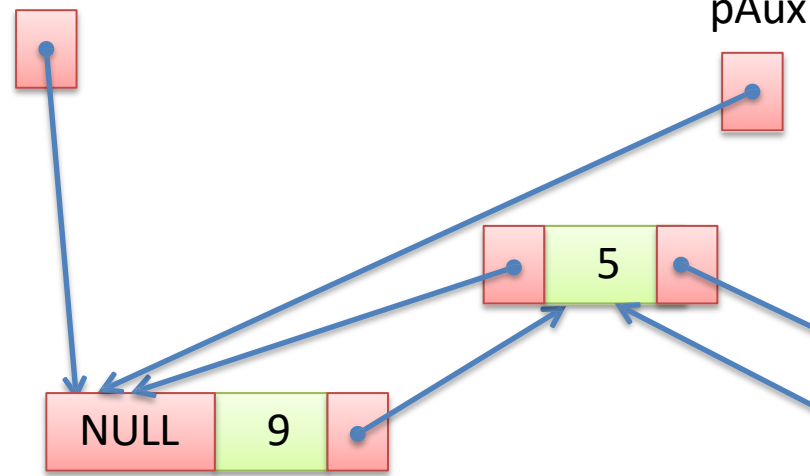
Lista Duplamente Encadeada

Inclusão na Calda

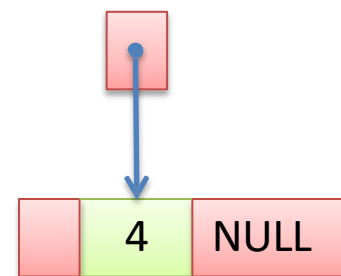
```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

Lista



pNovoNo



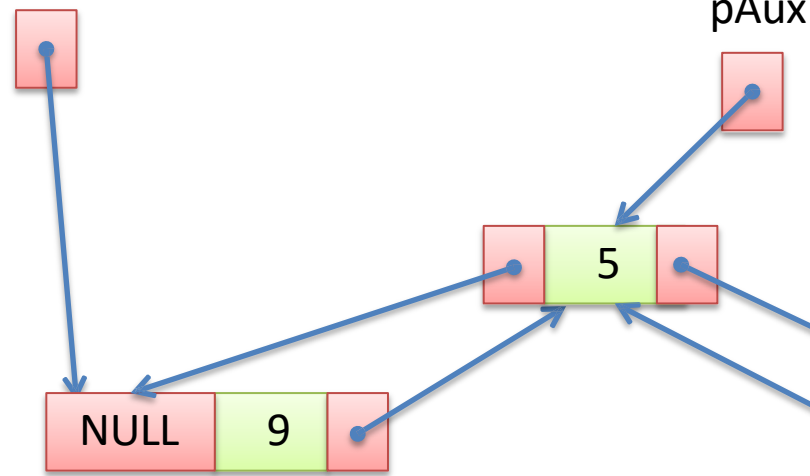
Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

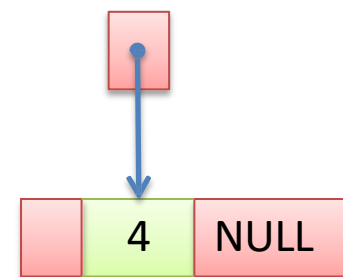
Lista



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

pNovoNo



Estrutura de Dados e Algoritmos

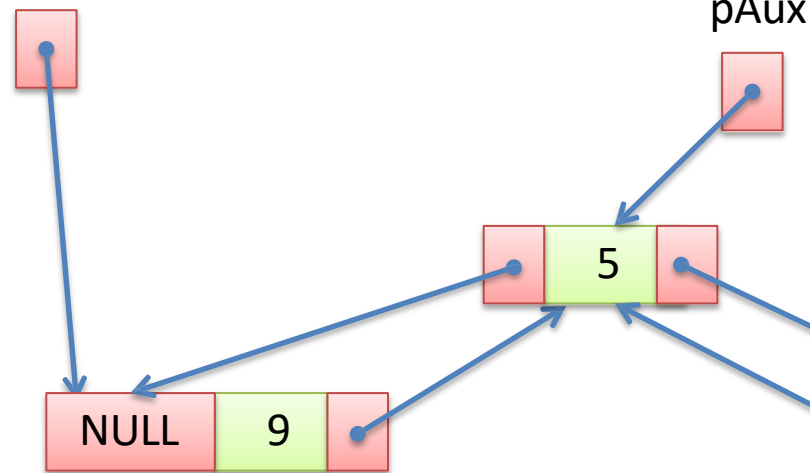
Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

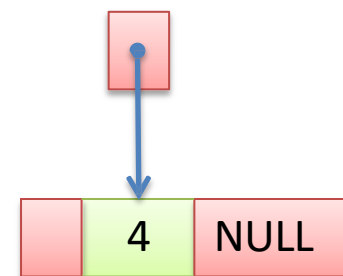
Lista



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

pNovoNo



Estrutura de Dados e Algoritmos

Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

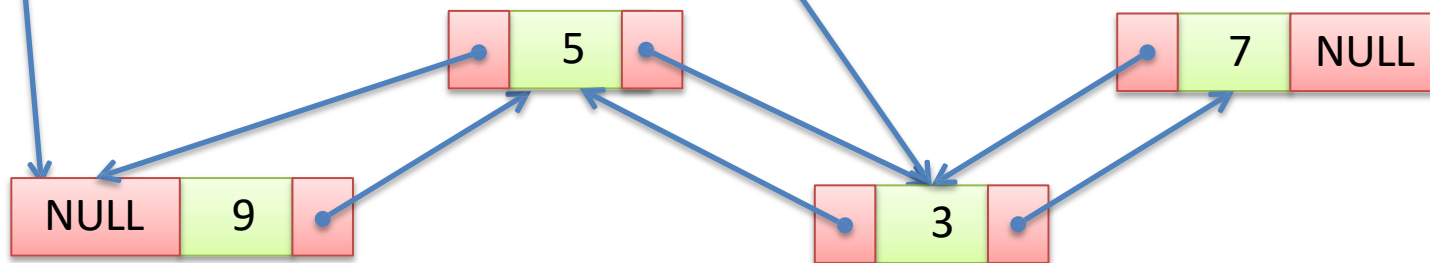
Lista



pAux



pNovoNo



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

Estrutura de Dados e Algoritmos

Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

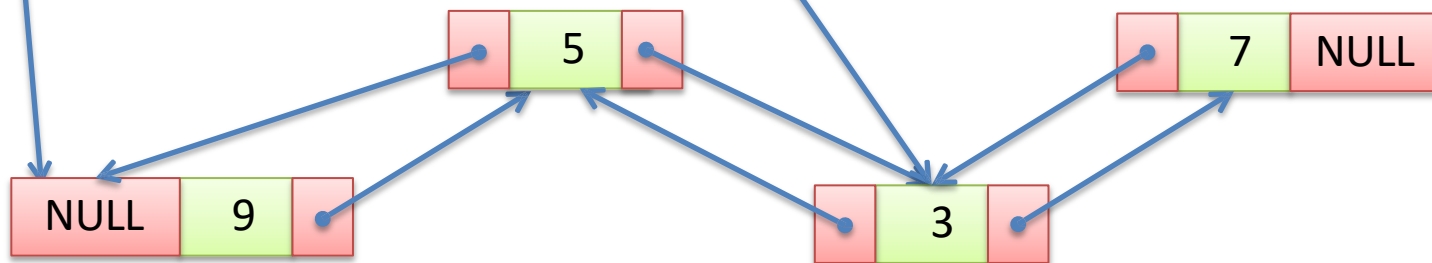
Lista



pAux



pNovoNo



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

Estrutura de Dados e Algoritmos

Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

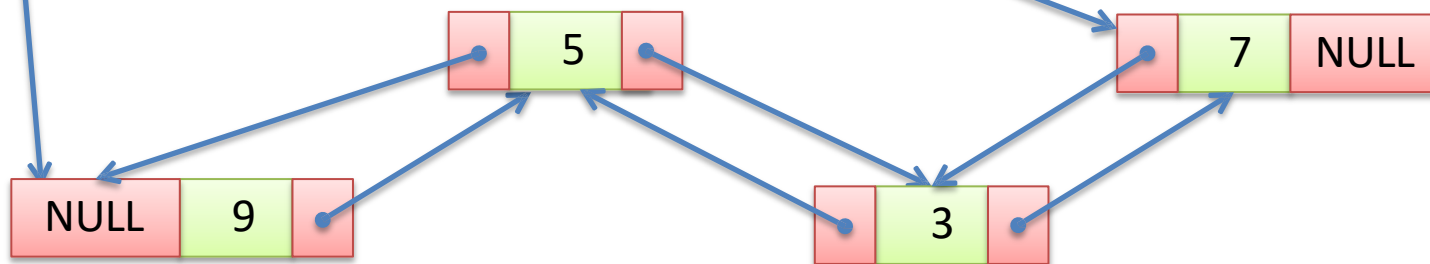
Lista



pAux



pNovoNo



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

Estrutura de Dados e Algoritmos

Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

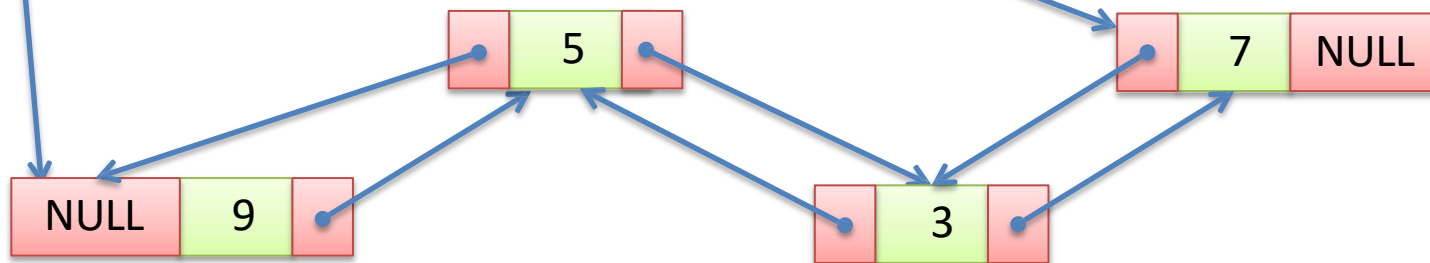
Lista



pAux



pNovoNo



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

Estrutura de Dados e Algoritmos

Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

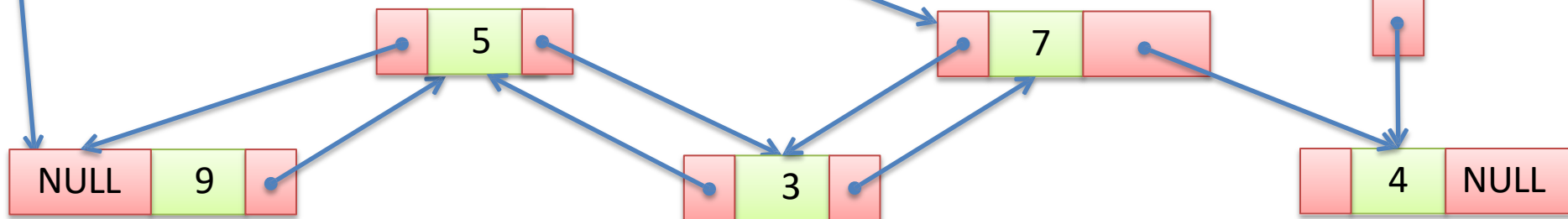
Lista



pAux



pNovoNo



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

Estrutura de Dados e Algoritmos

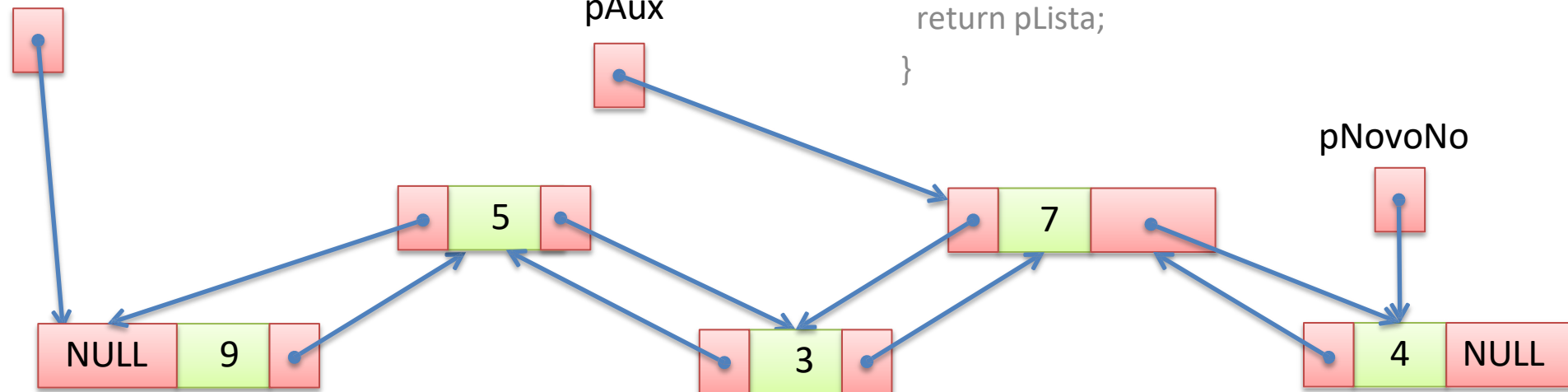
Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

Lista



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

Estrutura de Dados e Algoritmos

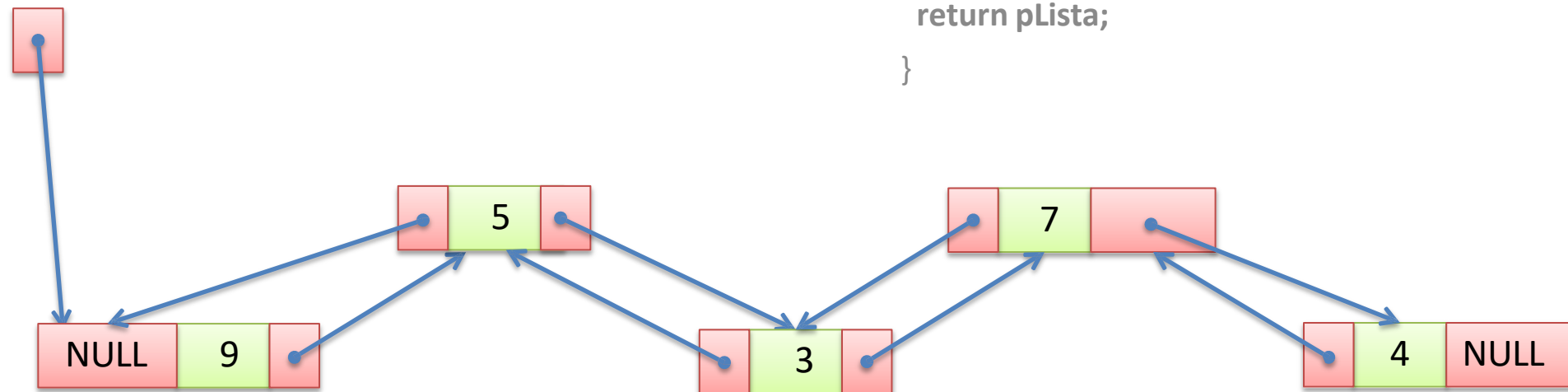
Lista Duplamente Encadeada

Inclusão na Calda

```
TNo *IncluiCalda(TNo *pLista, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pNovoNo->Prox = NULL;
    pAux = pLista;
    while (pAux->Prox != NULL)
        pAux = pAux->Prox;
    pAux->Prox = pNovoNo;
    pNovoNo->Ant = pAux;
    return pLista;
}
```

Lista



Listas Duplamente Encadeadas

Prof. MSc. Filipe Tôres

Estrutura de Dados e Algoritmos

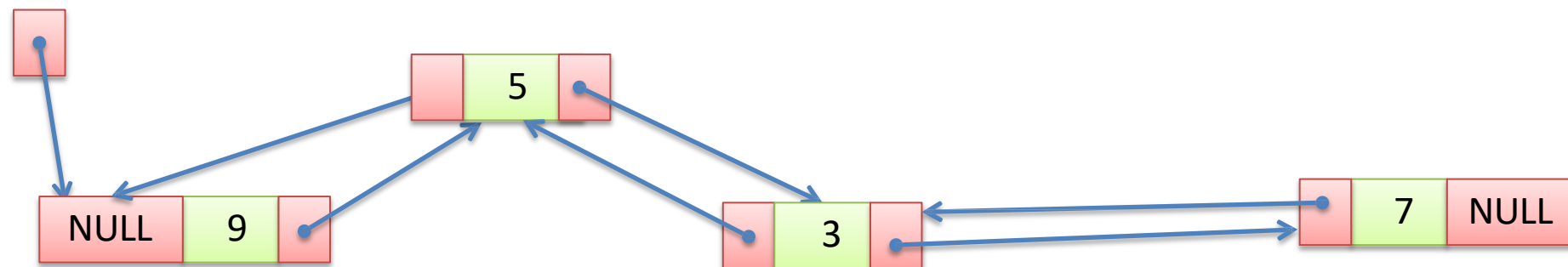
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo *IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista



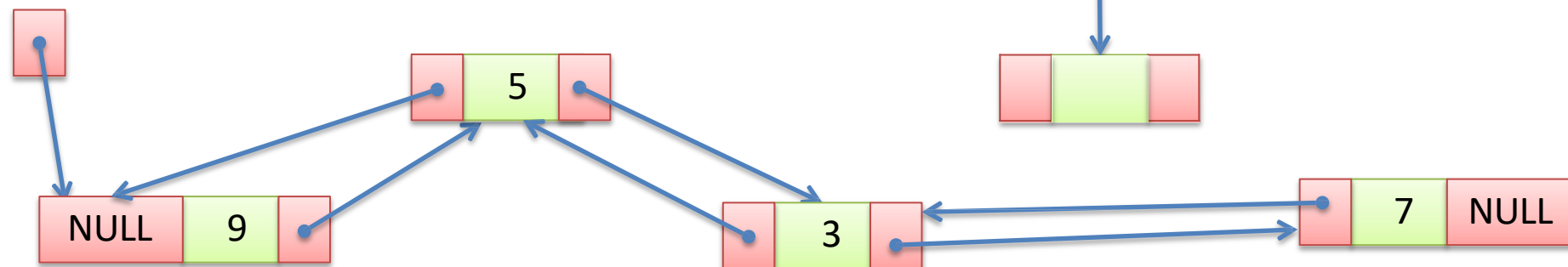
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista



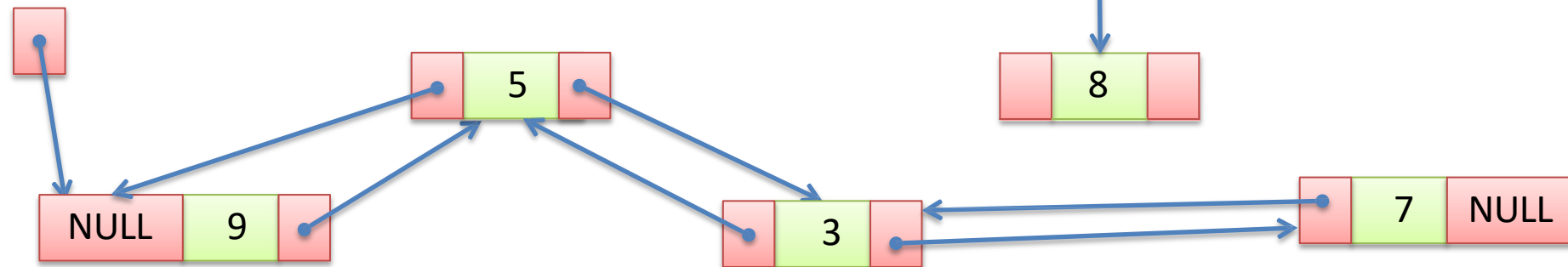
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista

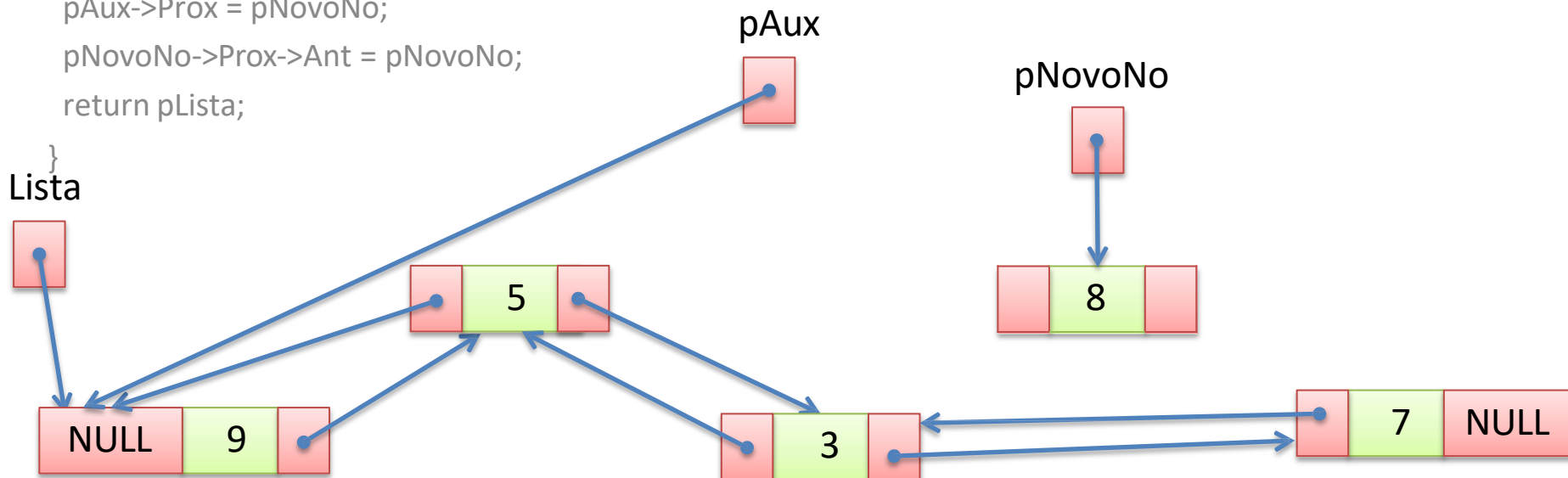


Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

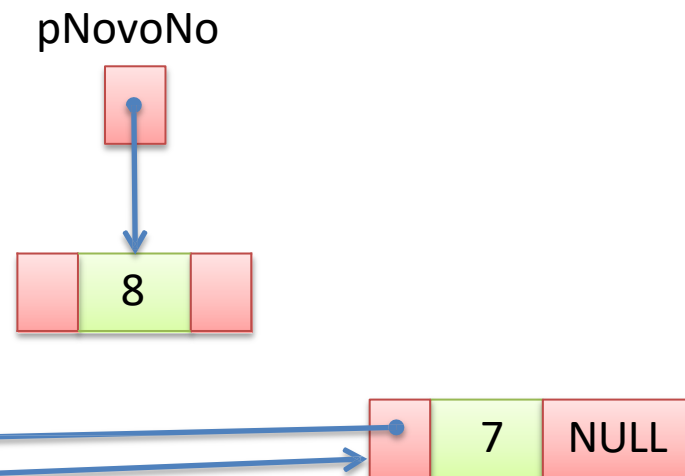
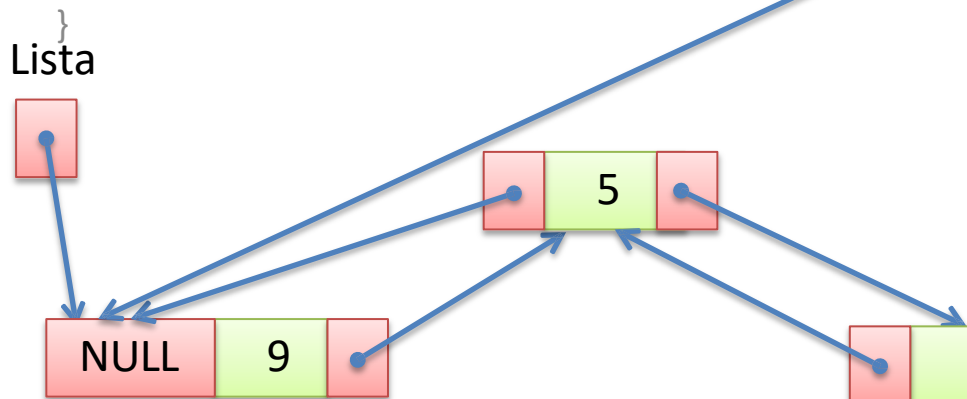


Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

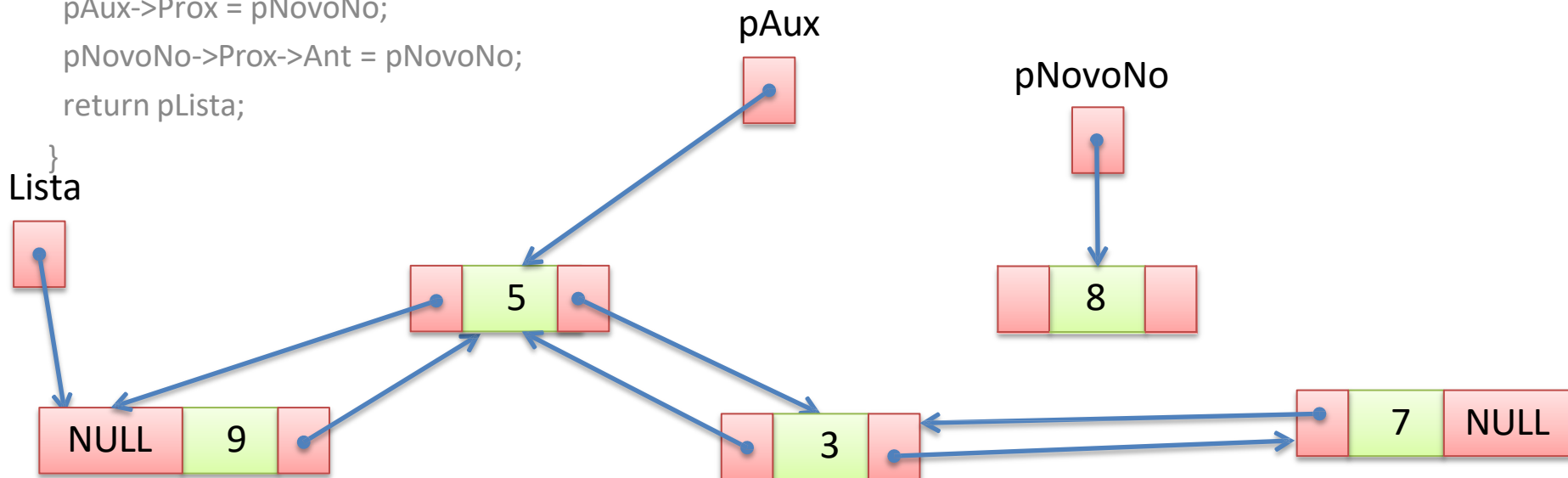


Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

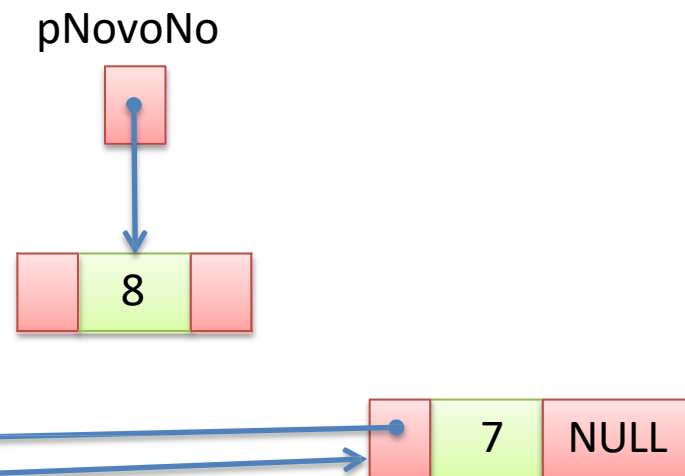
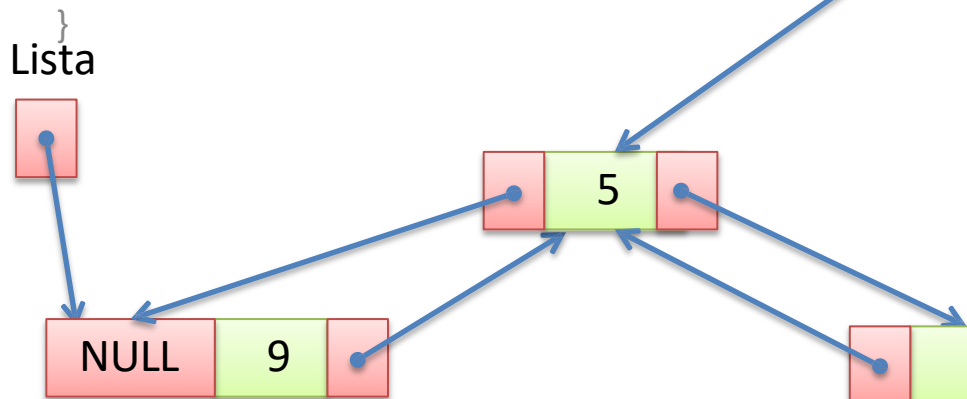


Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3



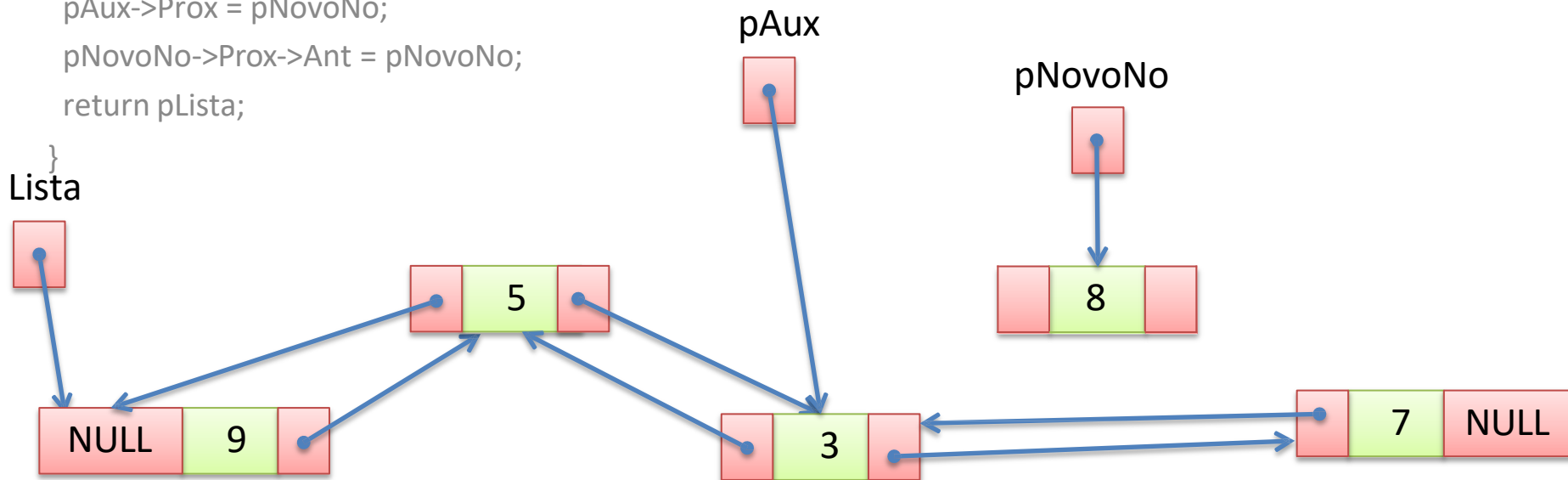
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista

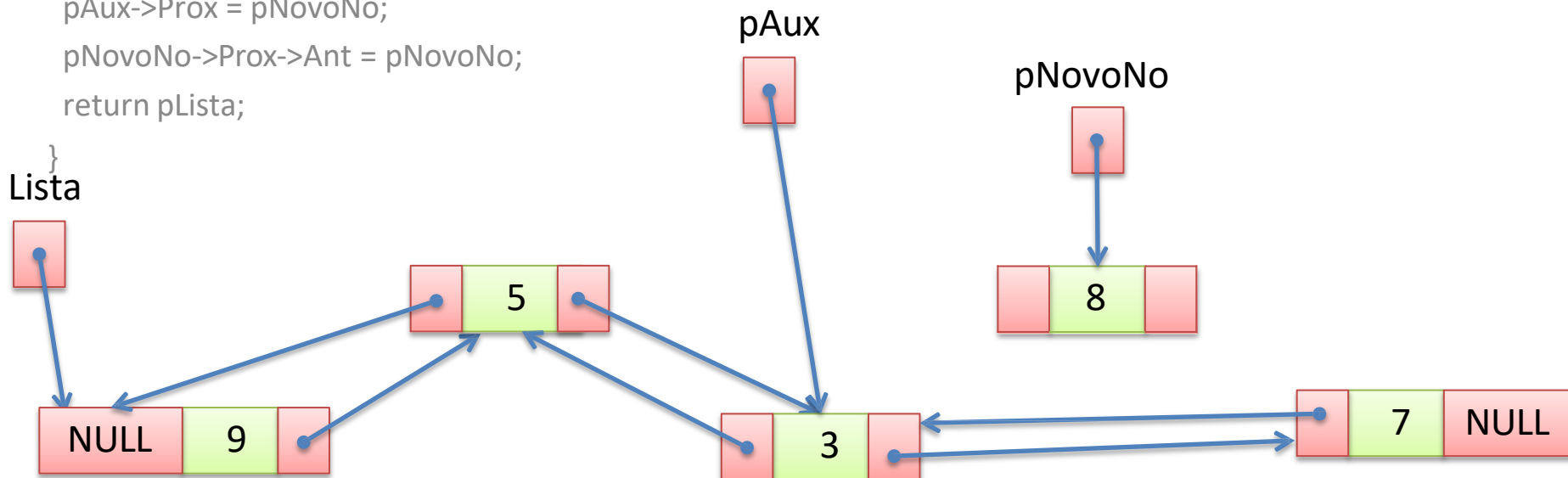


Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3



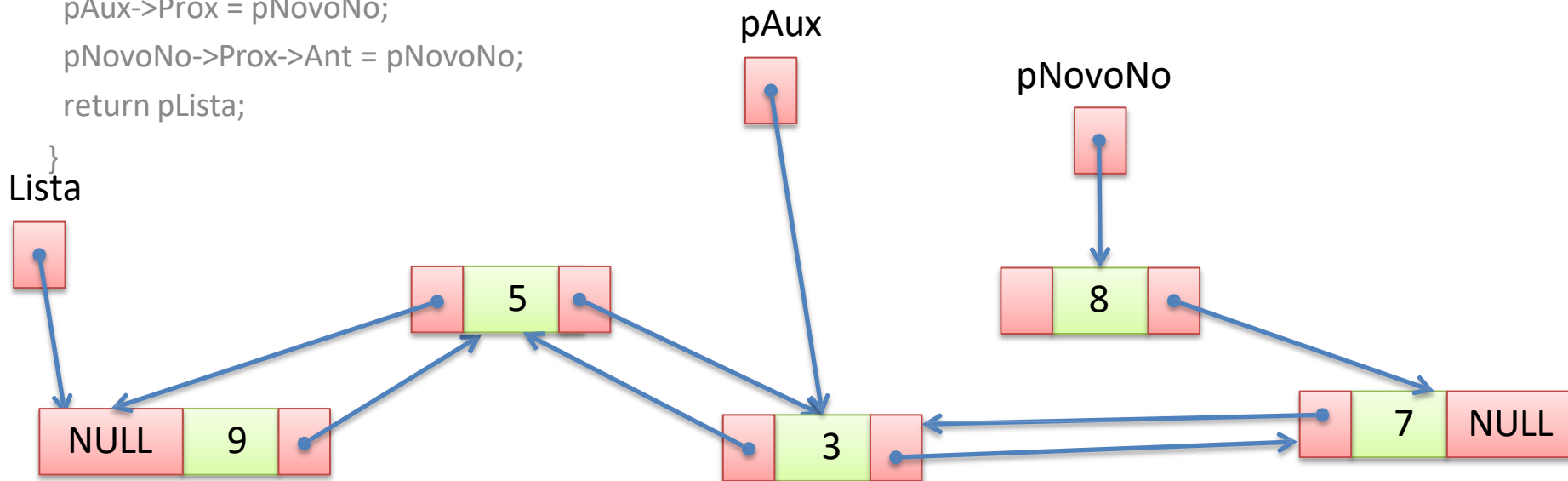
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista



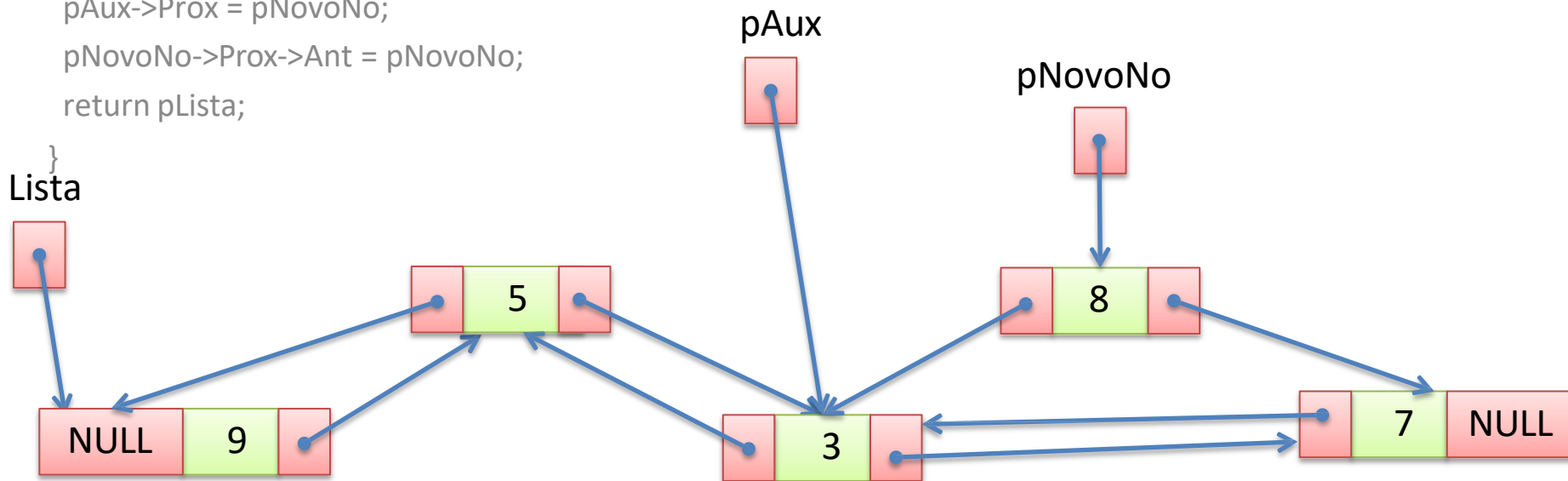
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista



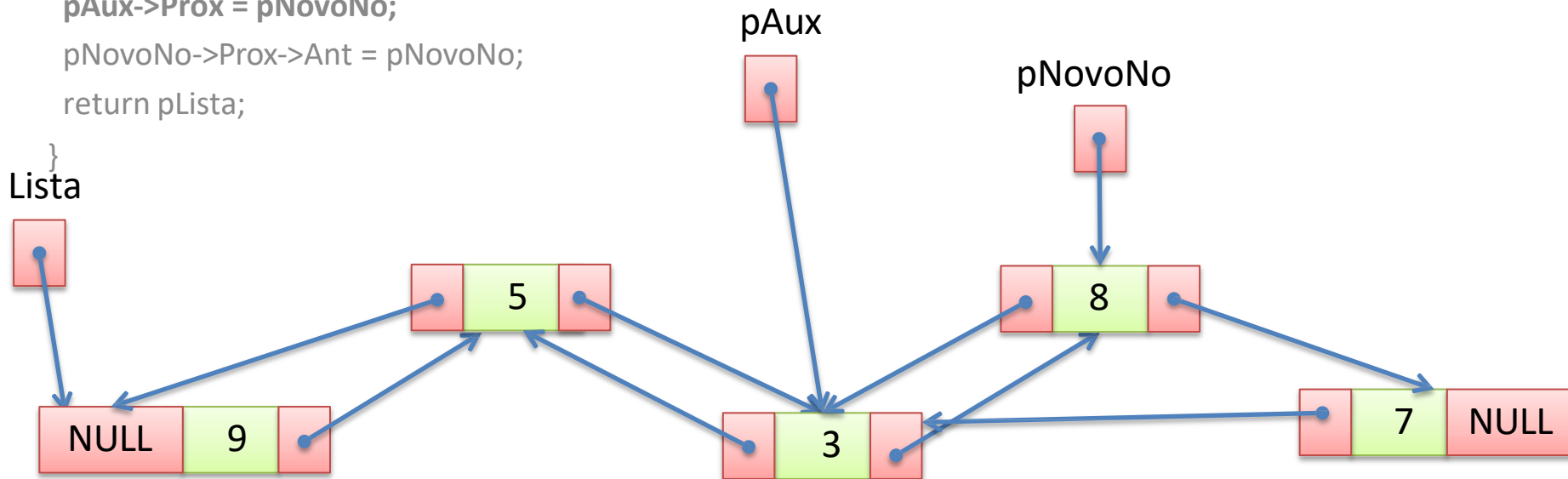
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovoNo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista



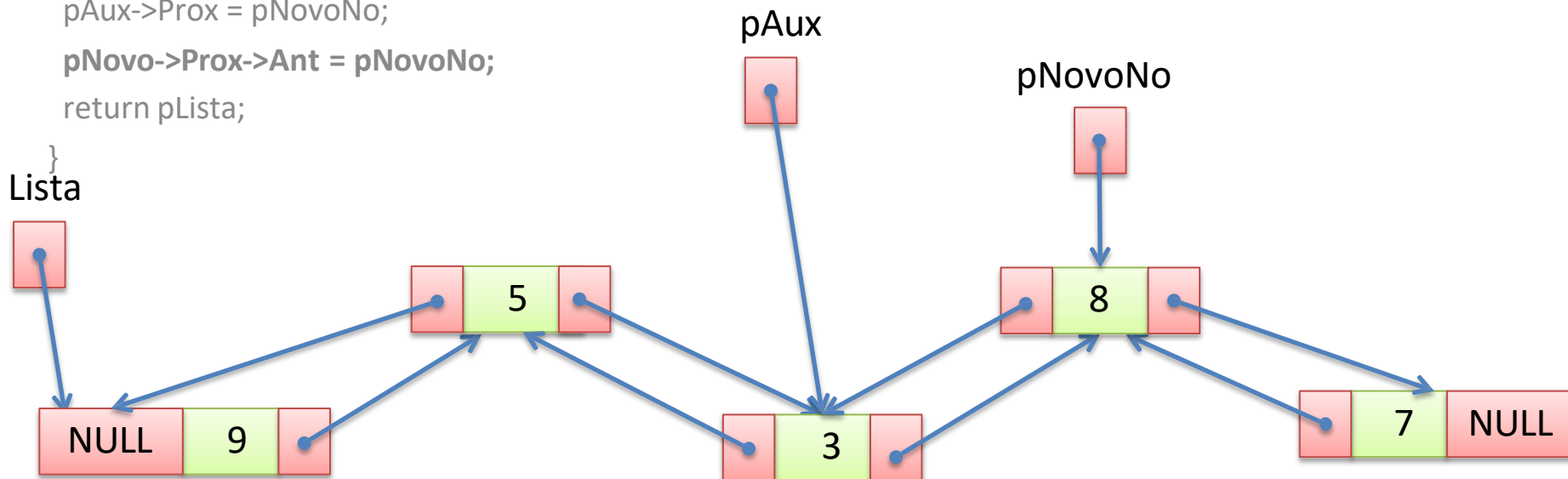
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

Lista



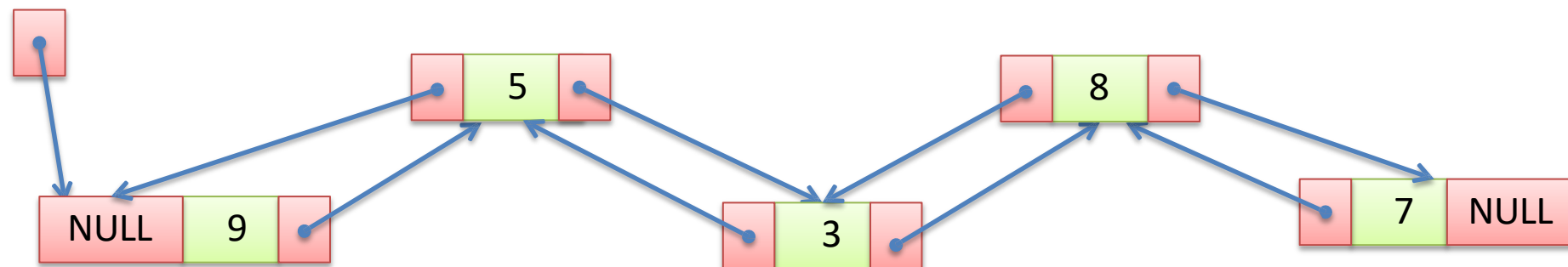
Lista Duplamente Encadeada Inclusão Depois de Uma Chave

```
TNo * IncluiDepois(TNo *pLista, int pChave, int pValor)
```

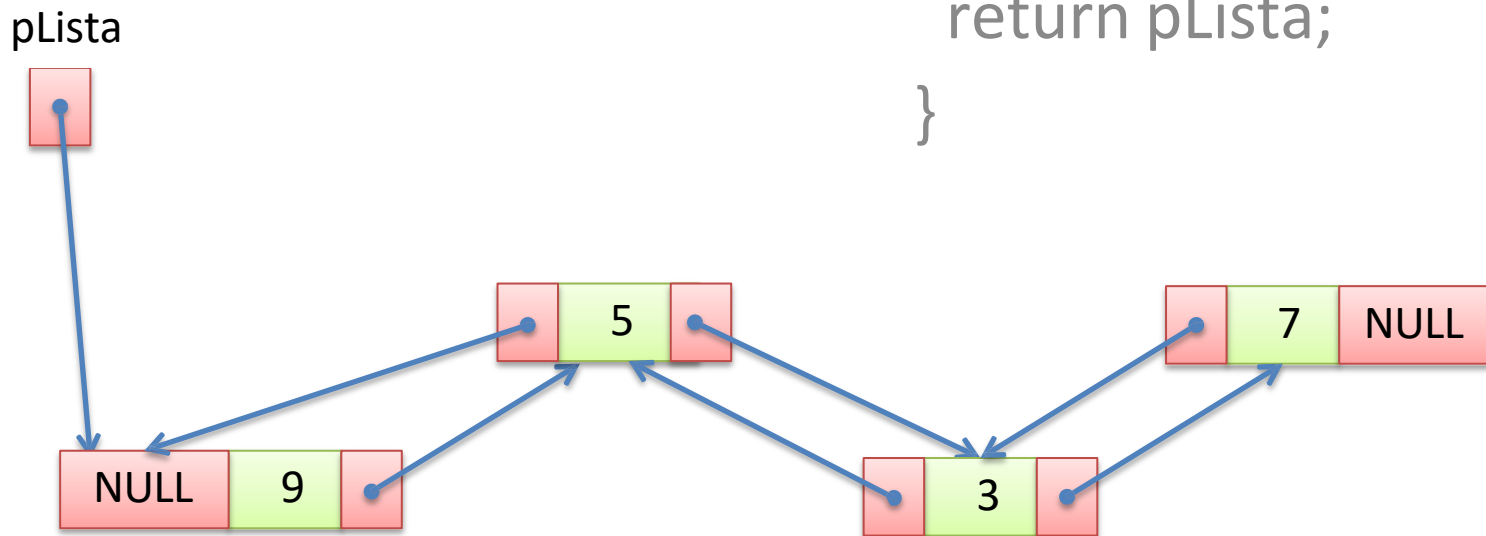
```
{
    TNo *pNovoNo, *pAux;
    pNovoNo = (TNo *) malloc(sizeof(TNo));
    pNovoNo->Numero = pValor;
    pAux = pLista;
    while (pAux->Prox->Valor != pChave)
        pAux = pAux->Prox;
    pNovoNo->Prox = pAux->Prox;
    pNovoNo->Ant = pAux;
    pAux->Prox = pNovoNo;
    pNovo->Prox->Ant = pNovoNo;
    return pLista;
}
```

pChave = 3

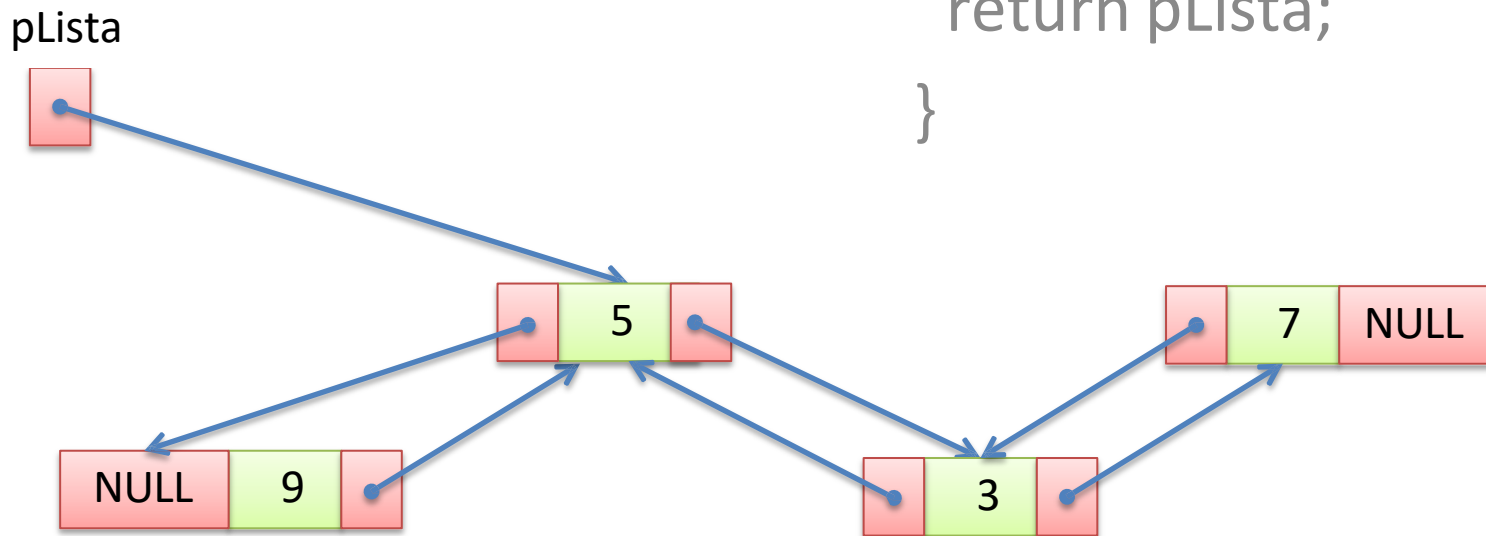
pLista



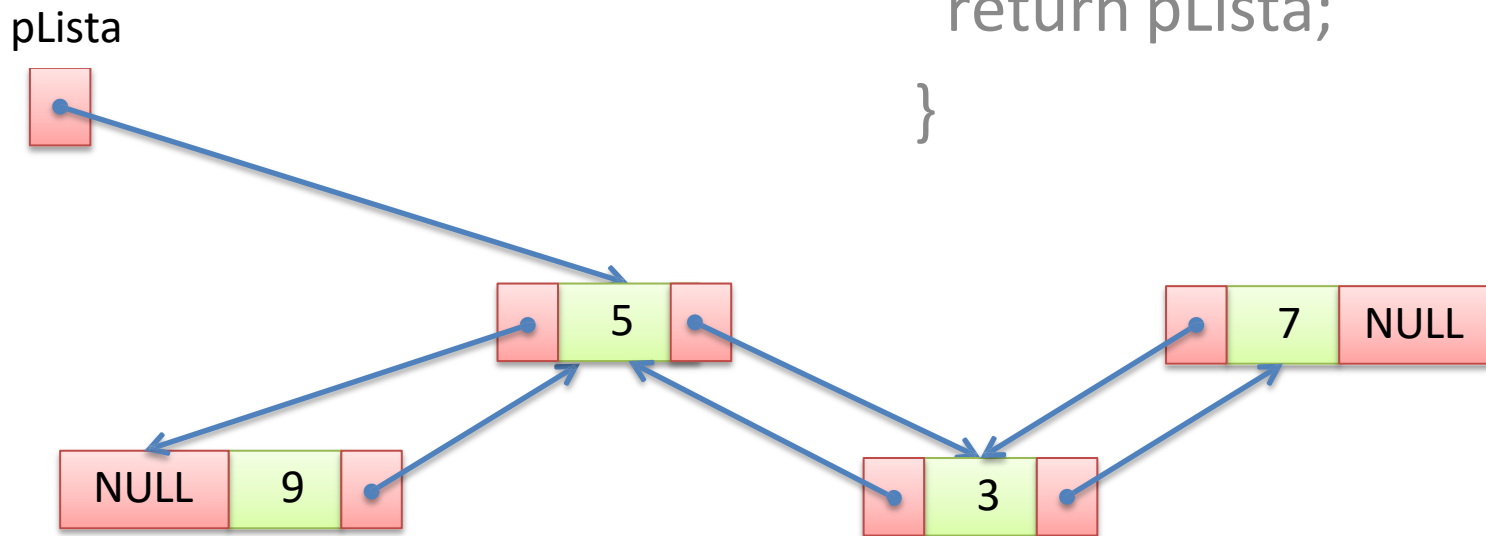
```
TNo *ExcluiCabeça(TNo *pLista)
{
    pLista = pLista->Prox;
    free(pLista->Ant);
    pLista->Ant = NULL;
    return pLista;
}
```



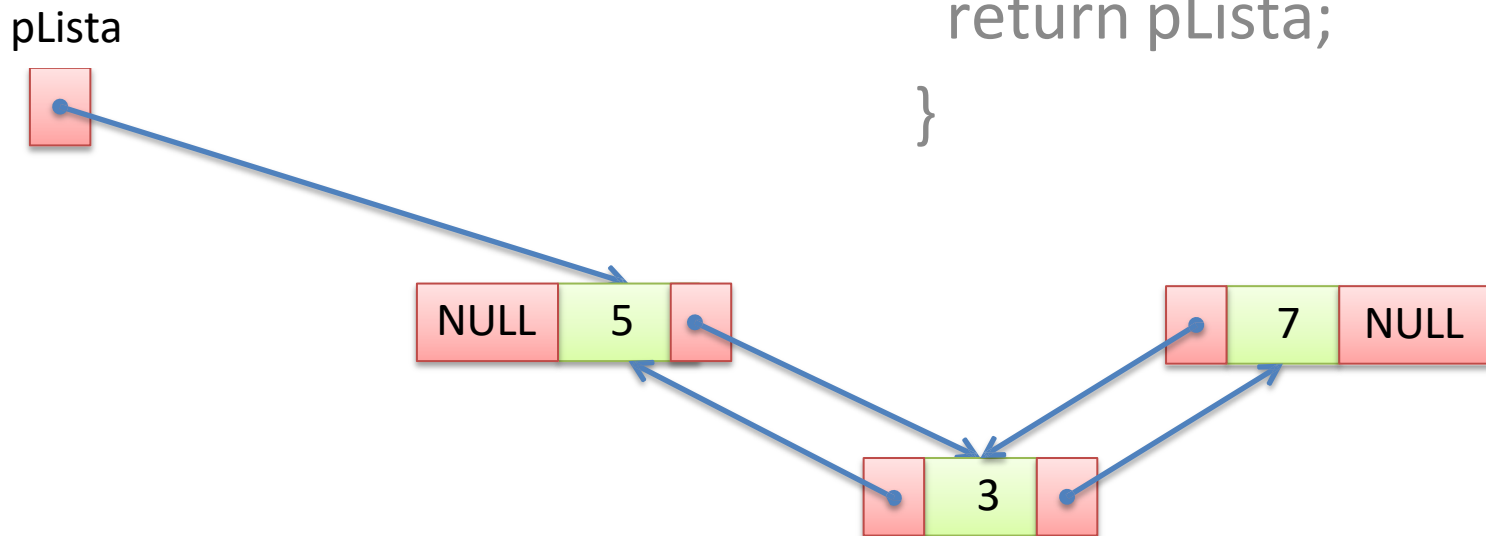
```
TNo *ExcluiCabeça(TNo *pLista)
{
    pLista = pLista->Prox;
    free(pLista->Ant);
    pLista->Ant = NULL;
    return pLista;
}
```



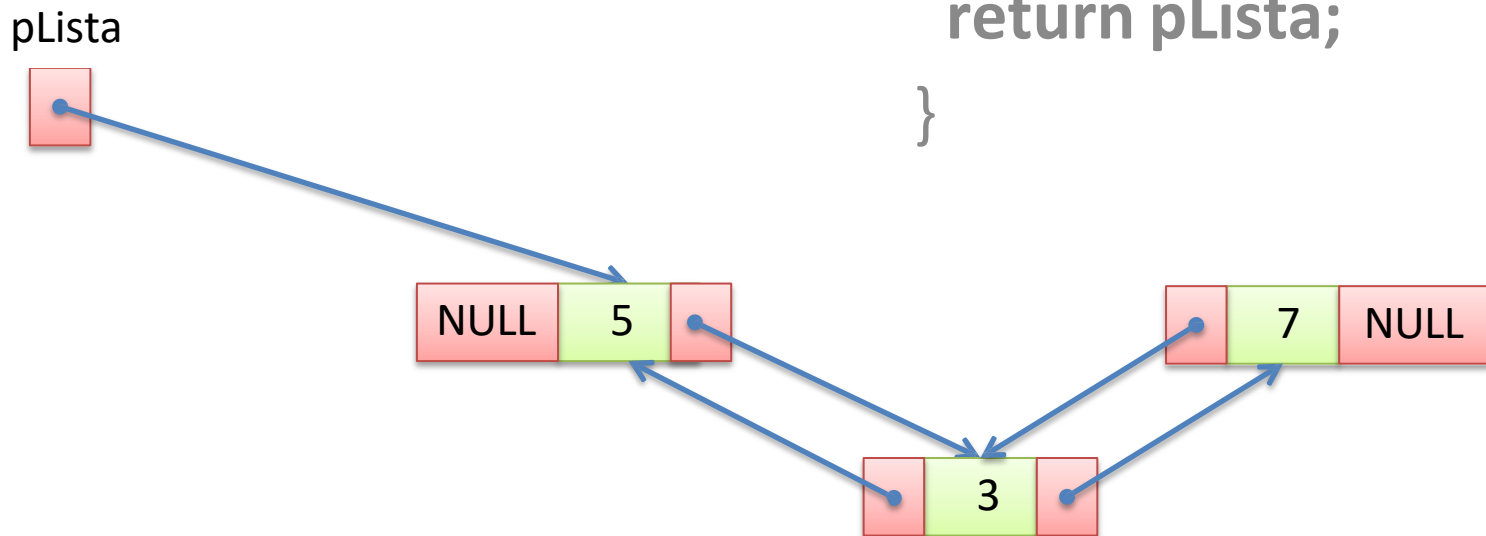
```
TNo *ExcluiCabeça(TNo *pLista)
{
    pLista = pLista->Prox;
    free(pLista->Ant);
    pLista->Ant = NULL;
    return pLista;
}
```



```
TNo *ExcluiCabeça(TNo *pLista)
{
    pLista = pLista->Prox;
    free(pLista->Ant);
    pLista->Ant = NULL;
    return pLista;
}
```



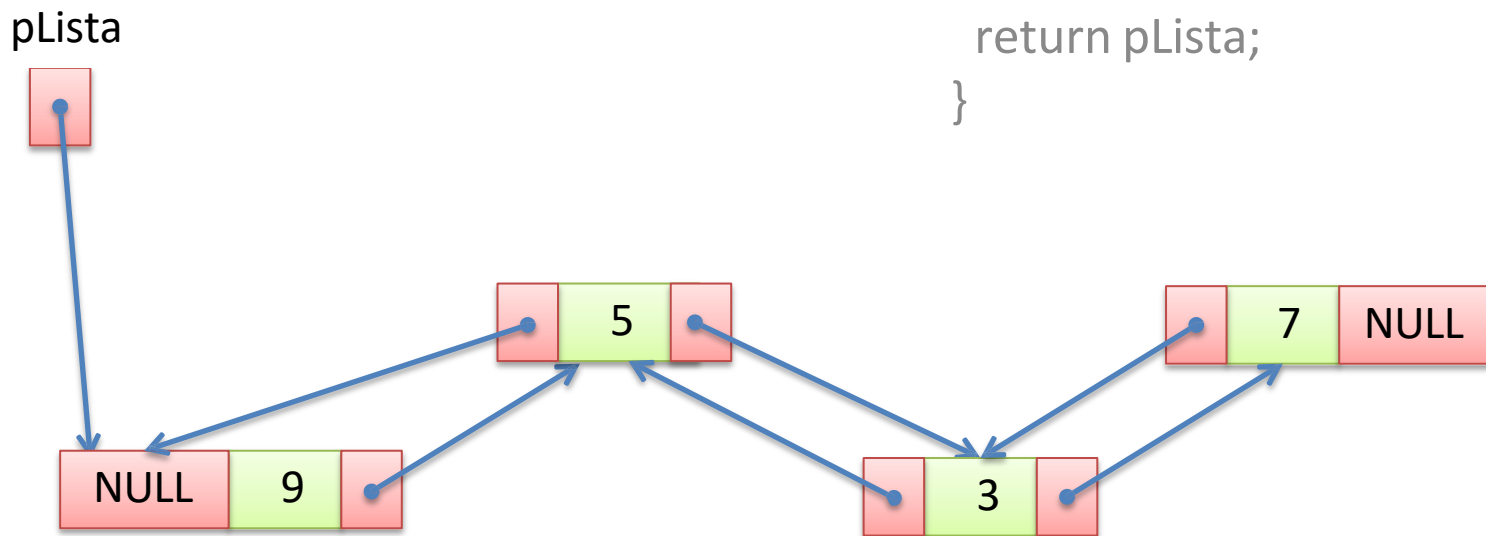
```
TNo *ExcluiCabeça(TNo *pLista)
{
    pLista = pLista->Prox;
    free(pLista->Ant);
    pLista->Ant = NULL;
    return pLista;
}
```



Lista Duplamente Encadeada

Exclusão na Calda

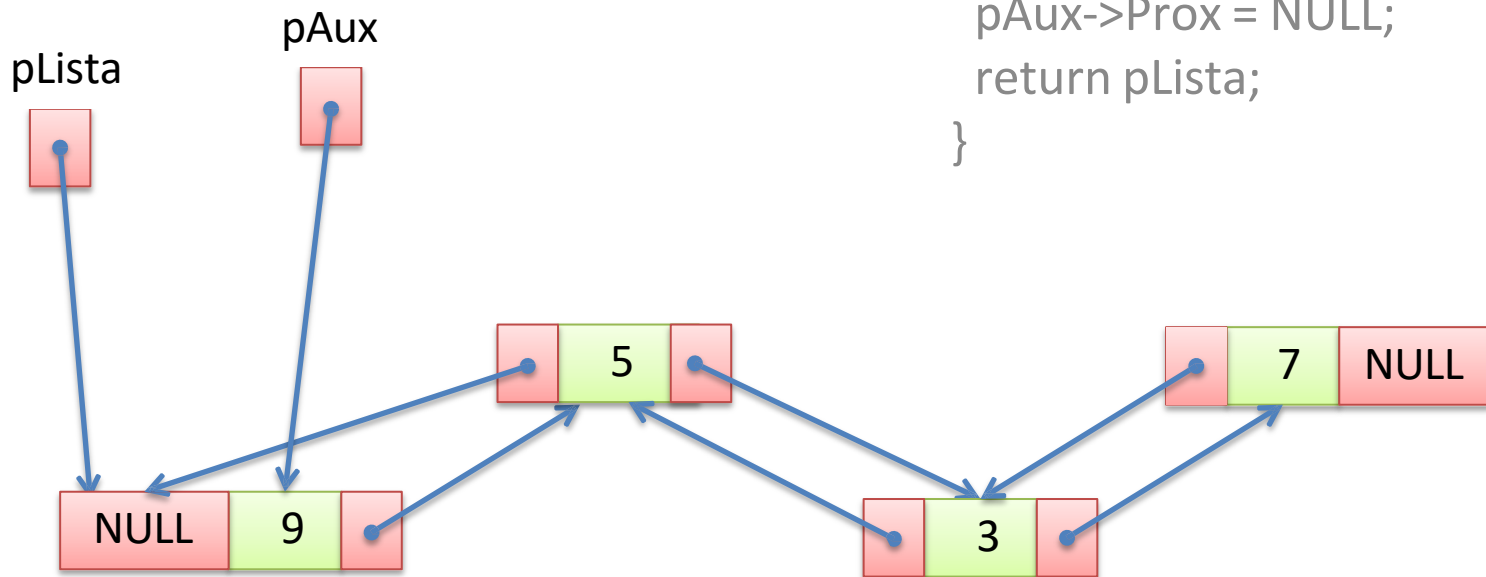
```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



Lista Duplamente Encadeada

Exclusão na Calda

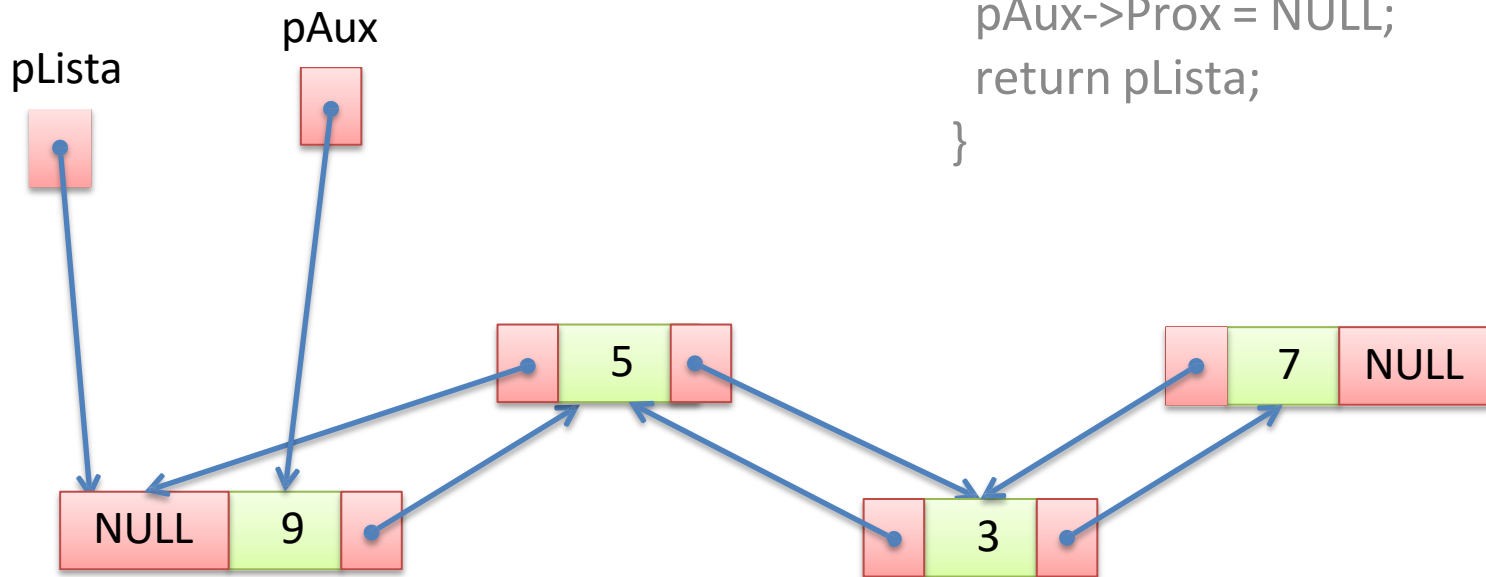
```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



Lista Duplamente Encadeada

Exclusão na Calda

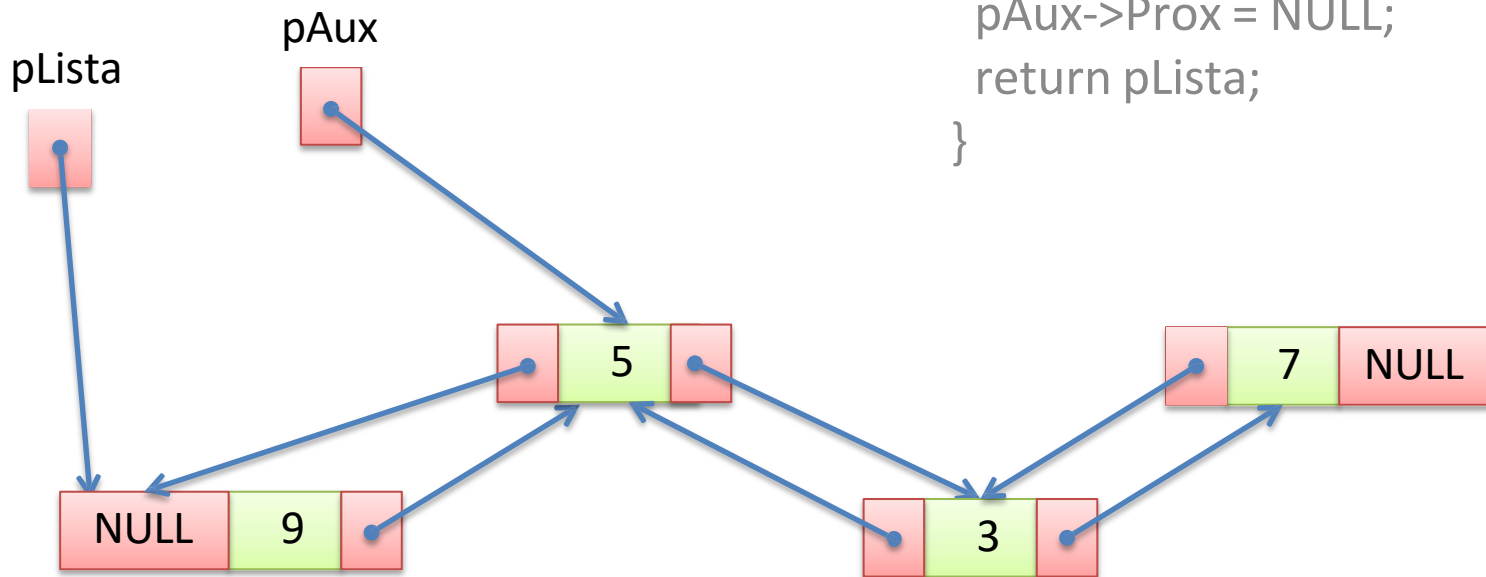
```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



Lista Duplamente Encadeada

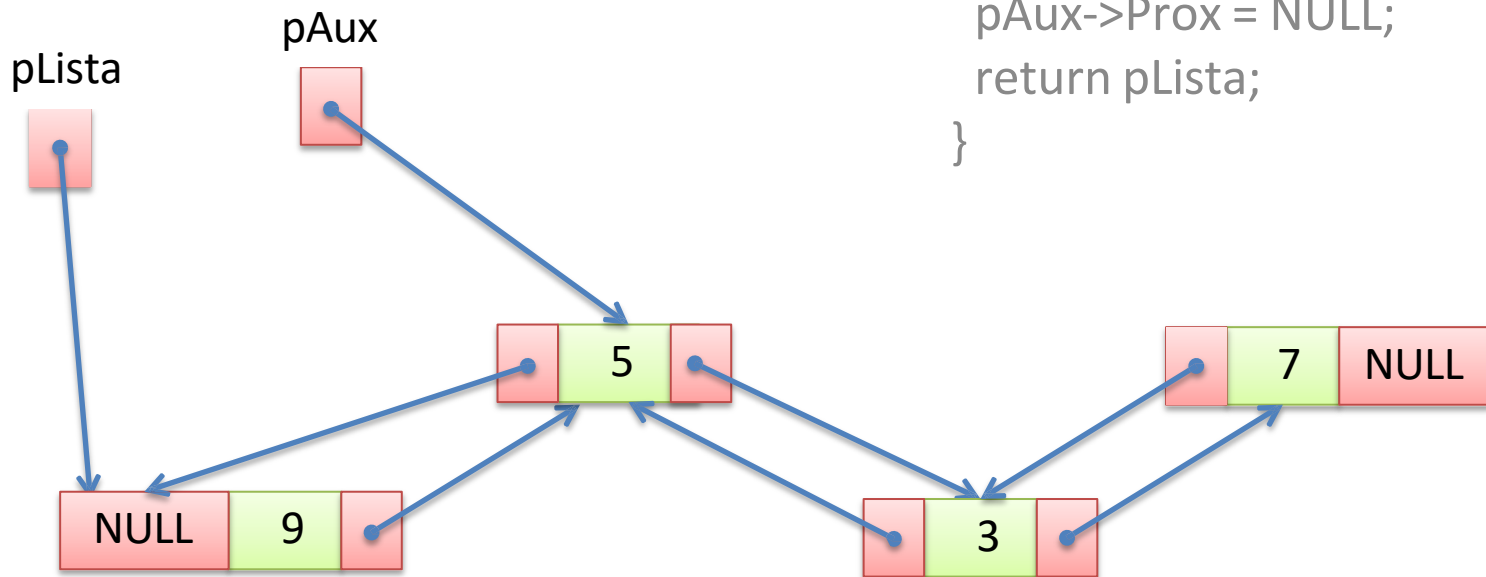
Exclusão na Calda

```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



Lista Duplamente Encadeada Exclusão na Calda

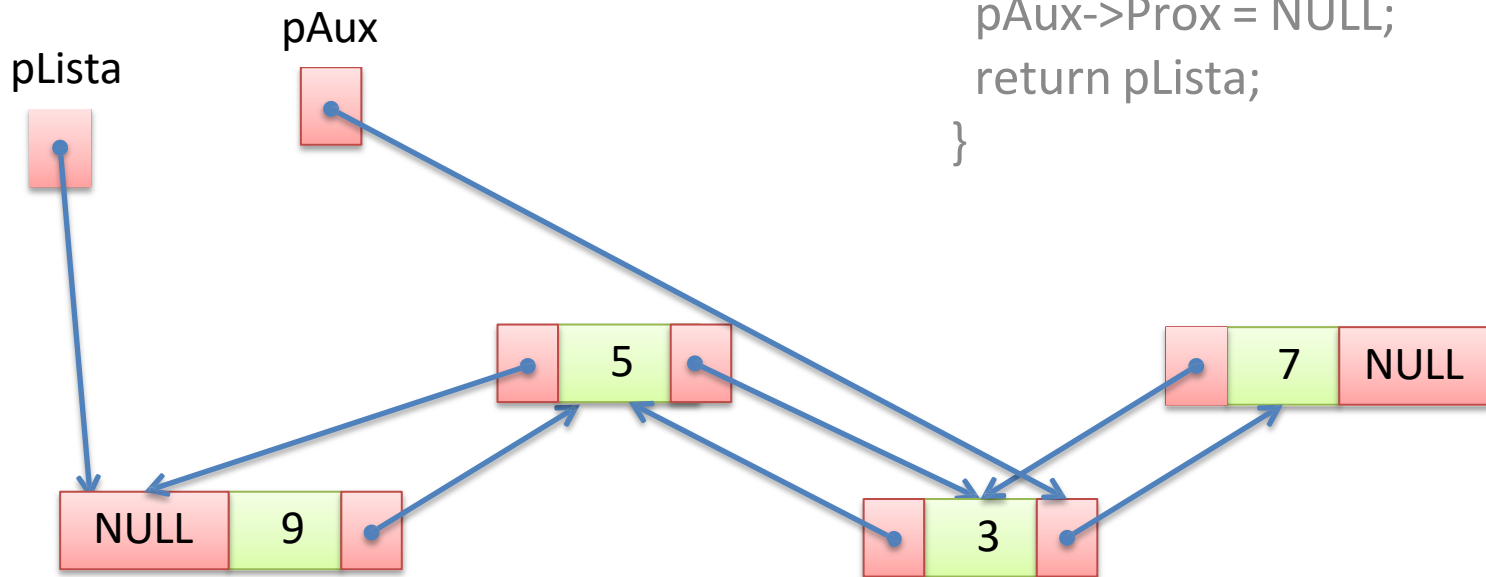
```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



Lista Duplamente Encadeada

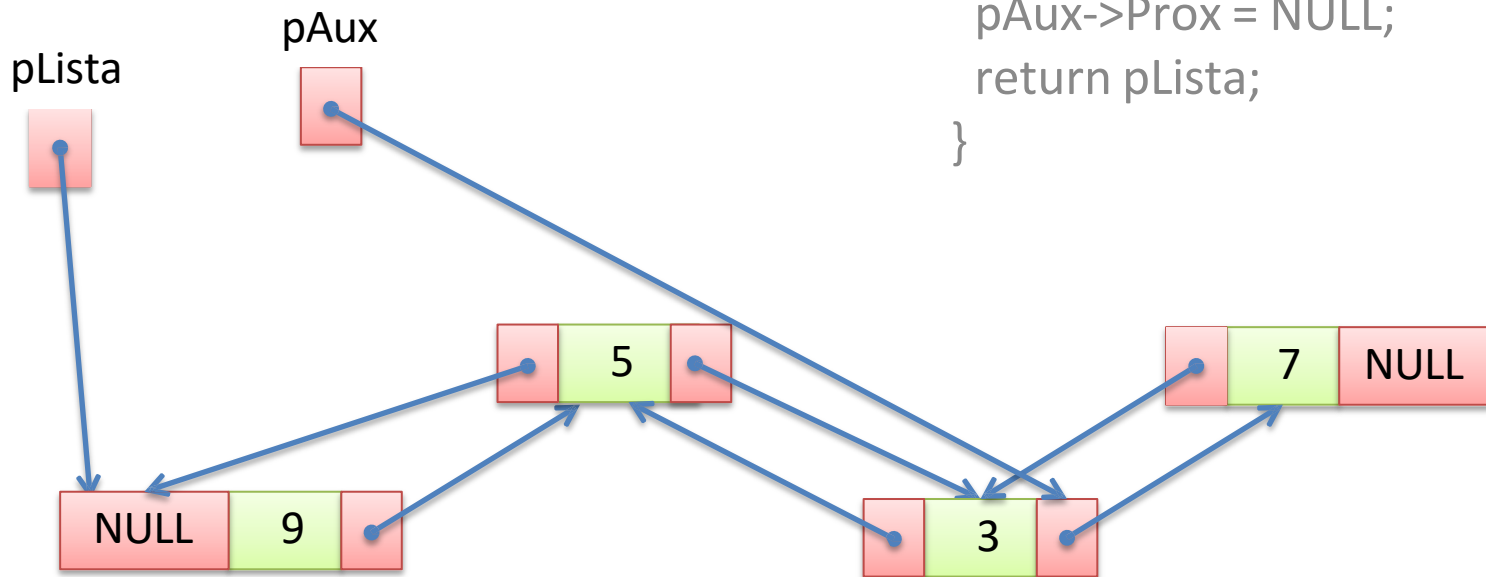
Exclusão na Calda

```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



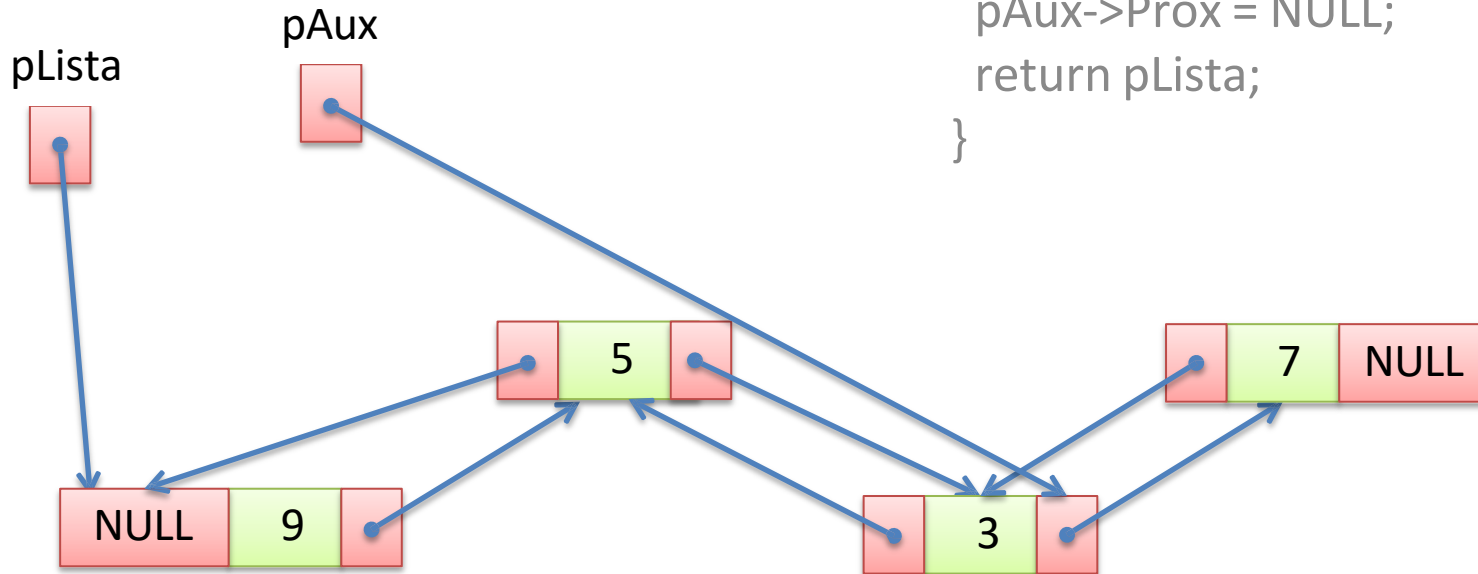
Lista Duplamente Encadeada Exclusão na Calda

```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



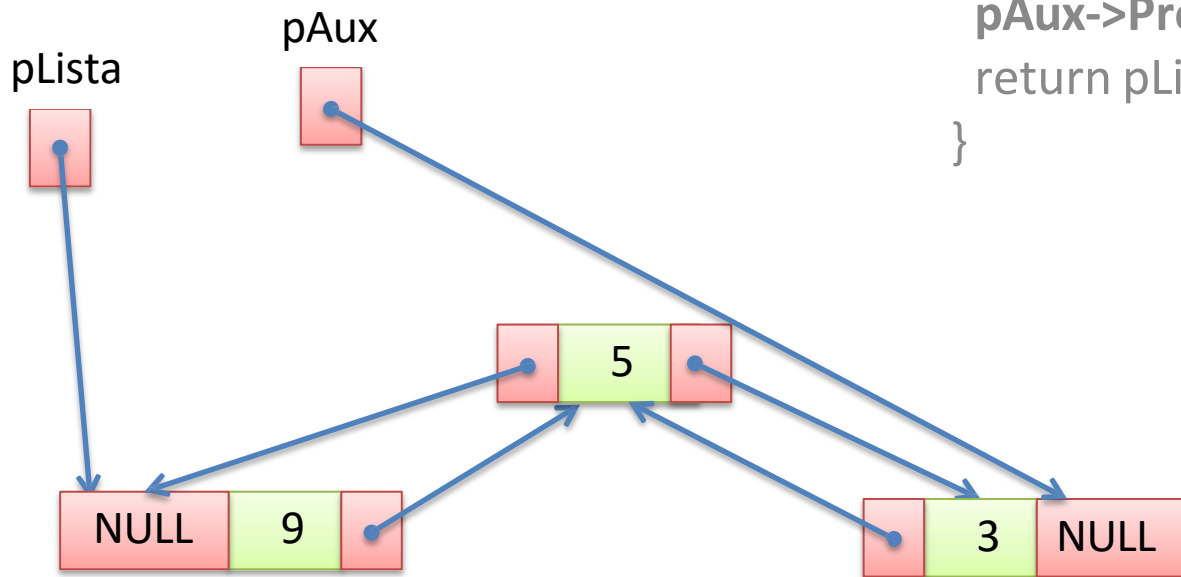
Lista Duplamente Encadeada Exclusão na Calda

```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



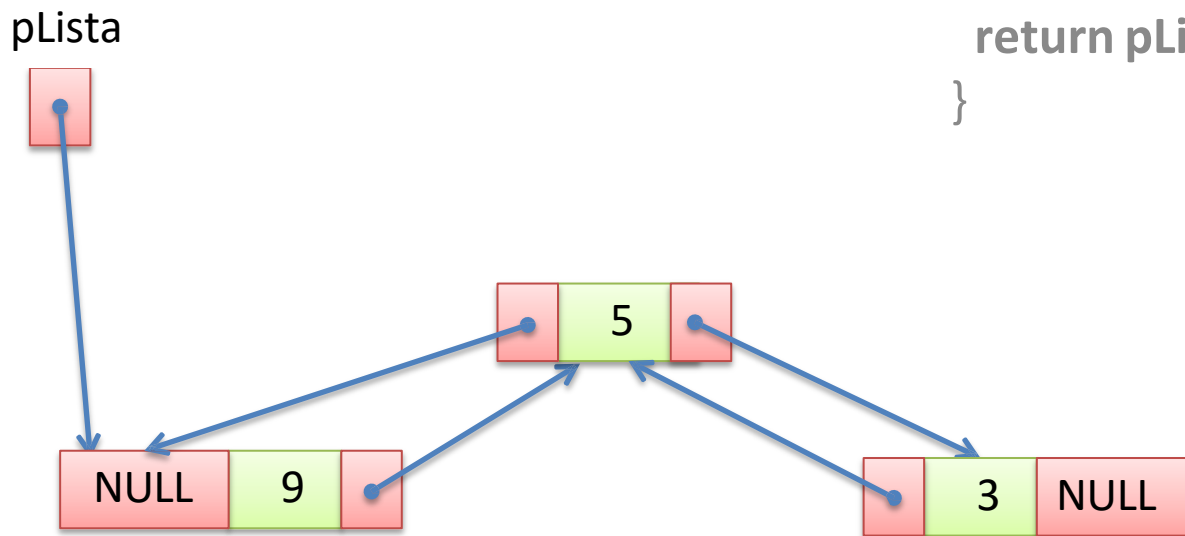
Lista Duplamente Encadeada Exclusão na Calda

```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



Lista Duplamente Encadeada Exclusão na Calda

```
TNo *ExcluiCalda(TNo *pLista)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Prox->Prox != NULL)
        pAux = pAux->Prox;
    free(pAux->Prox);
    pAux->Prox = NULL;
    return pLista;
}
```



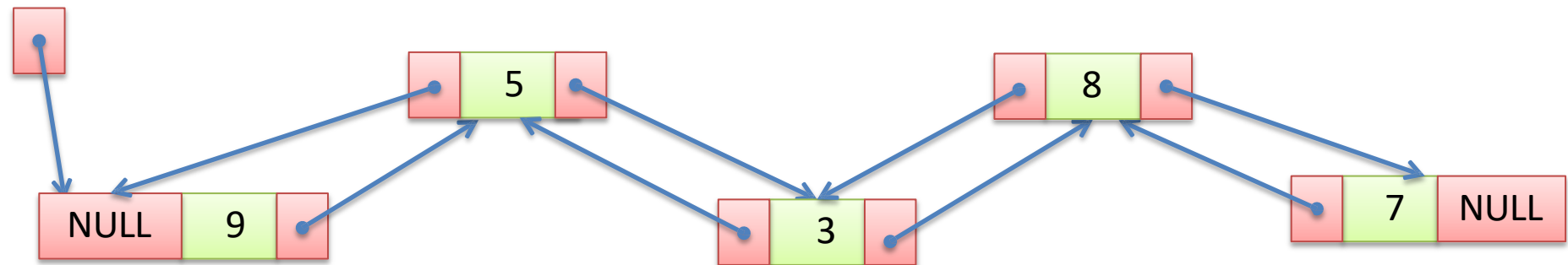
Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

pLista

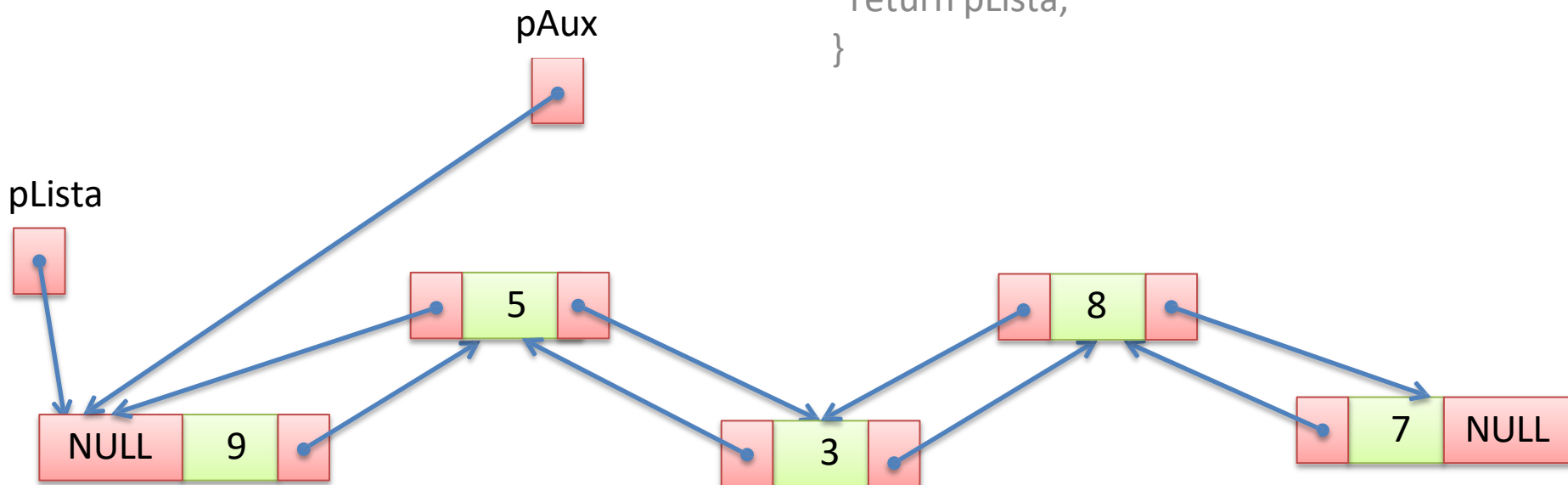


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

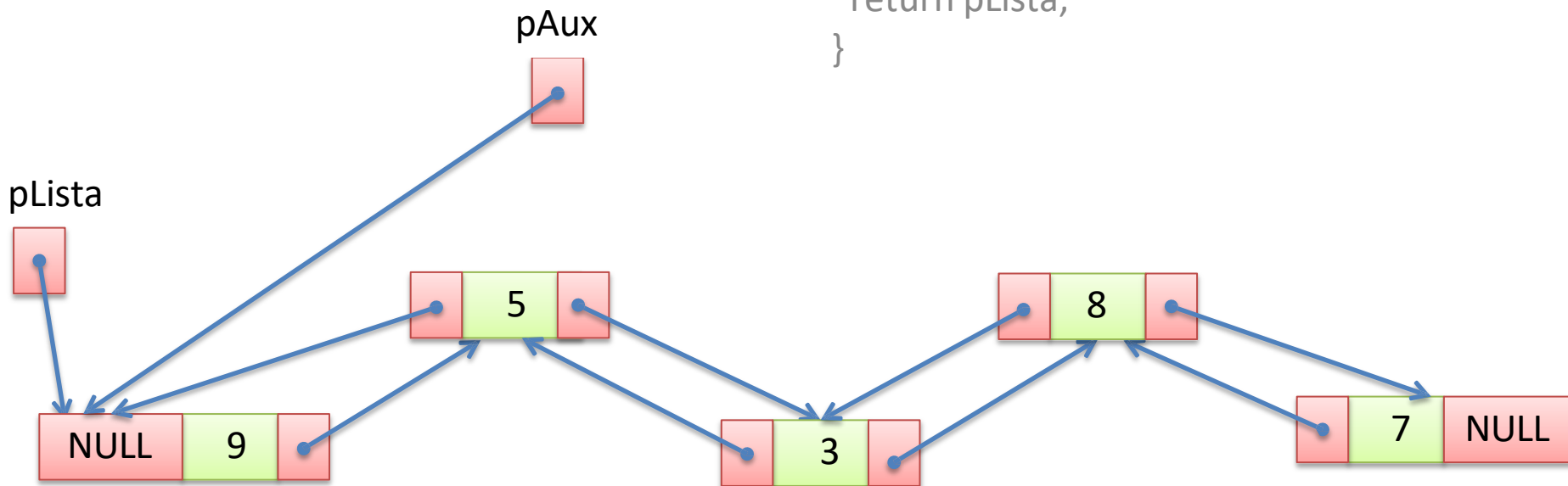


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

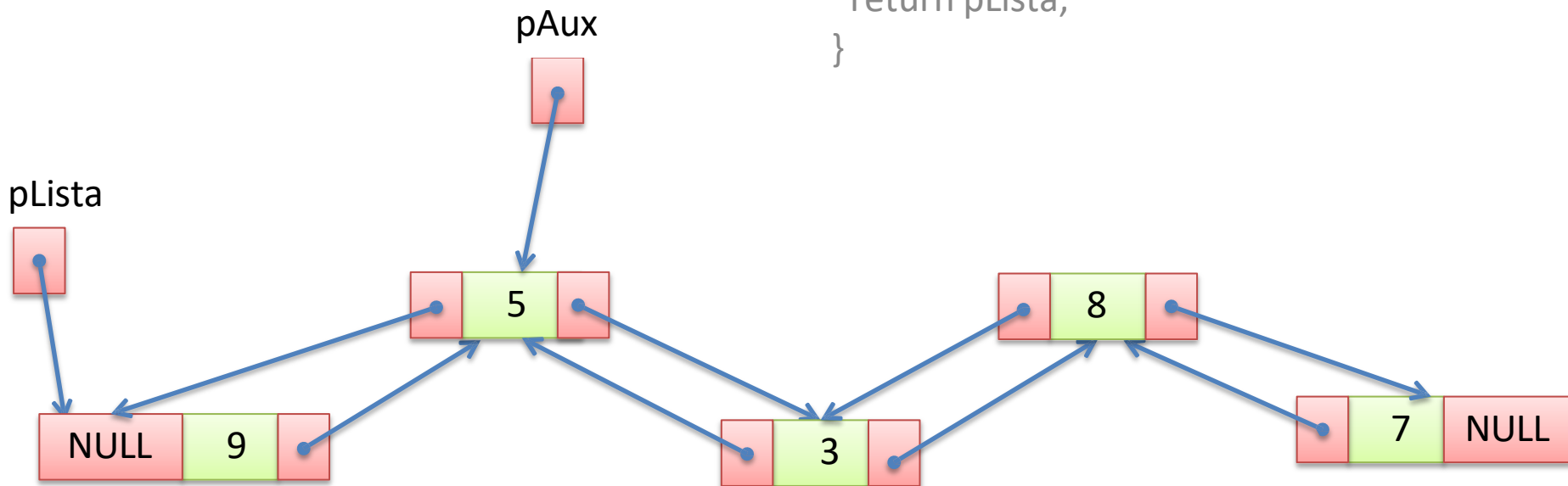


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

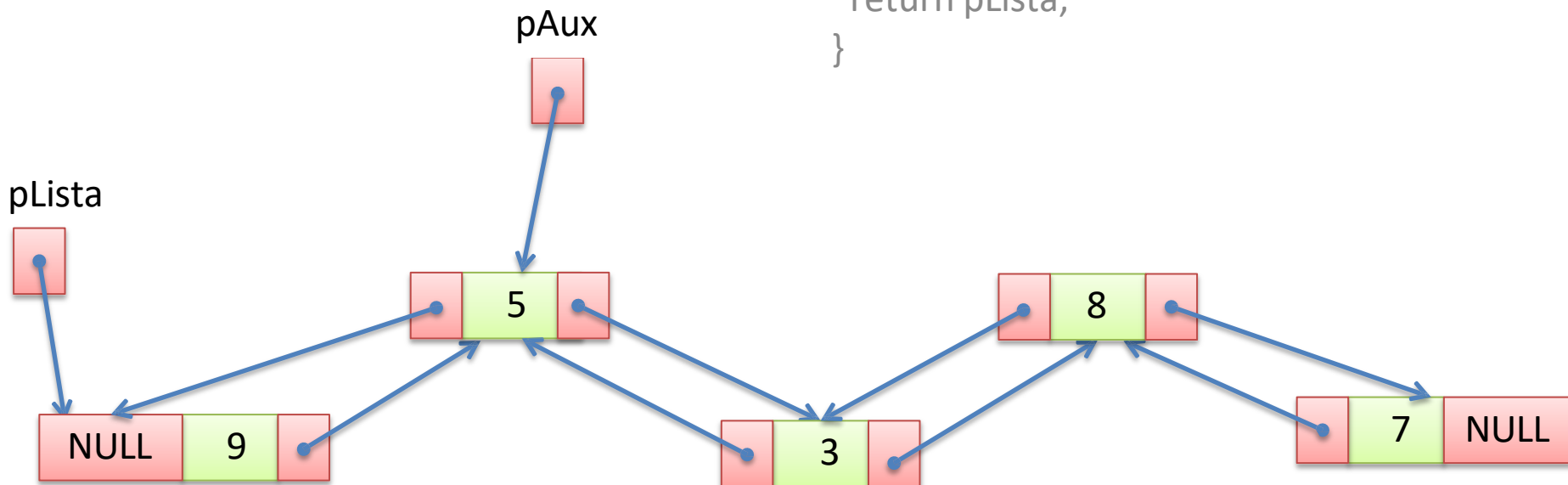


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

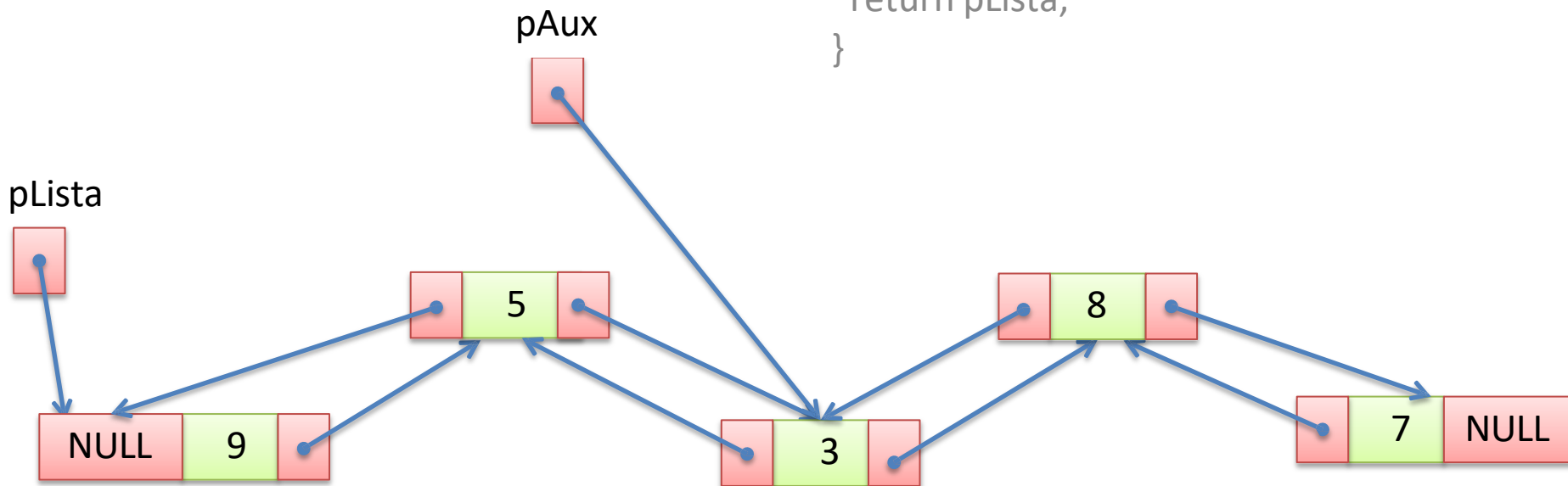


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

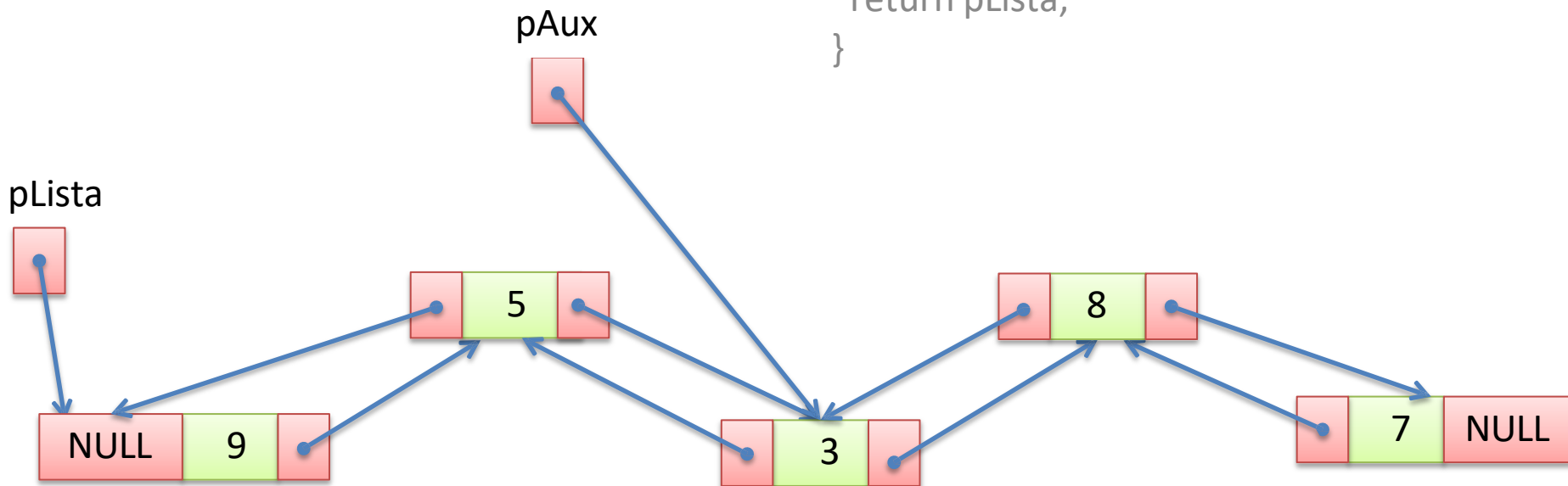


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

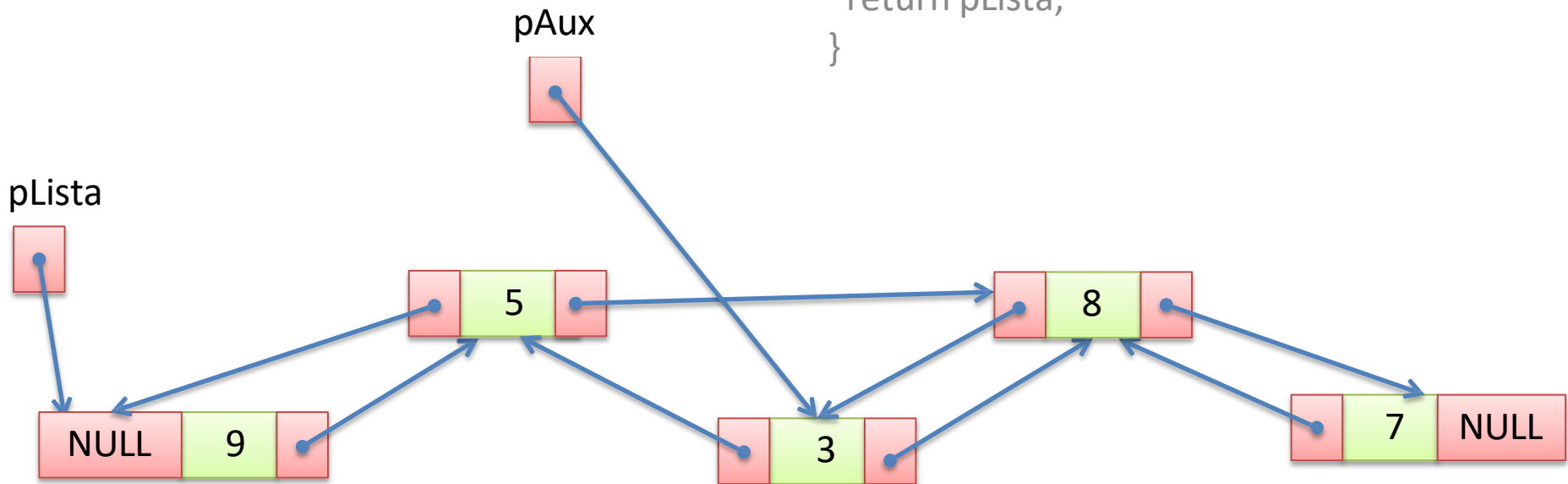


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

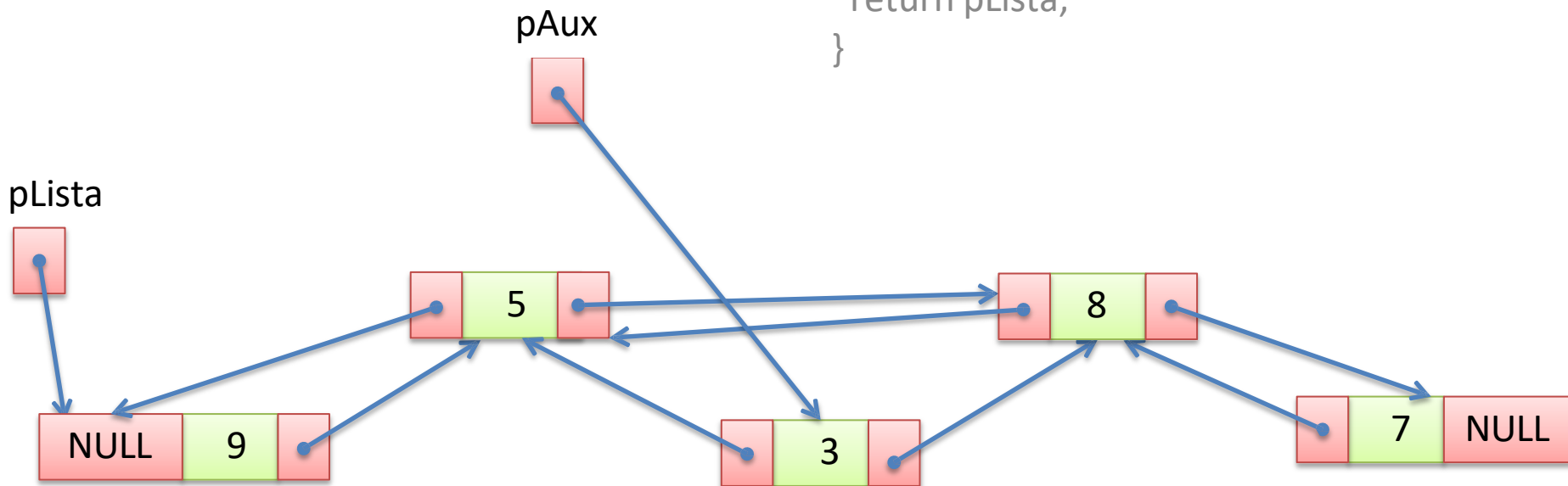


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

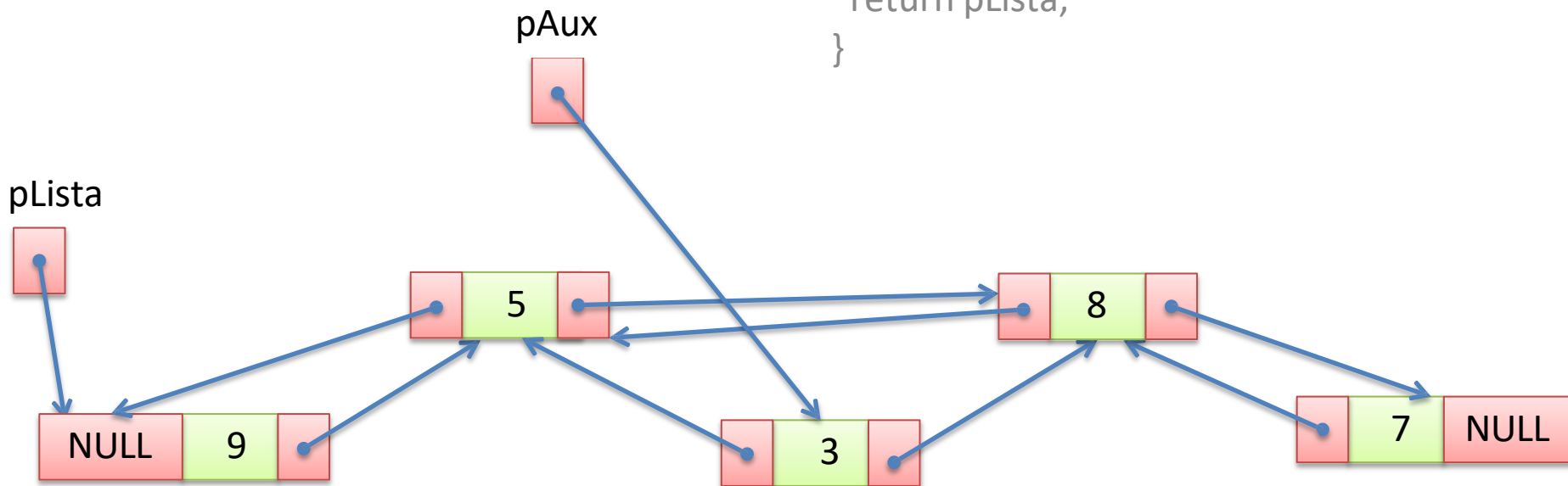


Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```



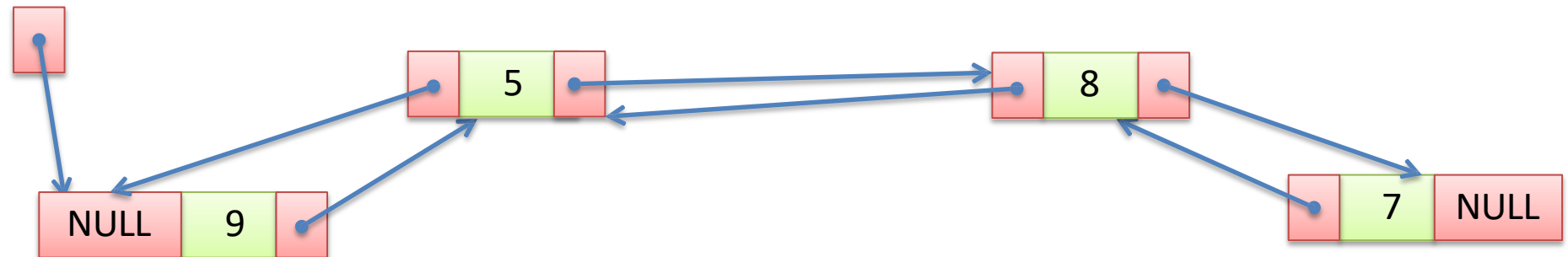
Lista Duplamente Encadeada

Exclusão de Uma Chave

pChave = 3

```
TNo *ExcluiChave(TNo *pLista, int pChave)
{
    TNo *pAux;
    pAux = pLista;
    while (pAux->Numero != pChave)
        pAux = pAux->Prox;
    pAux->Ant->Prox = pAux->Prox;
    pAux->Prox->Ant = pAux->Ant;
    free(pAux);
    return pLista;
}
```

pLista



```
struct TNo
```

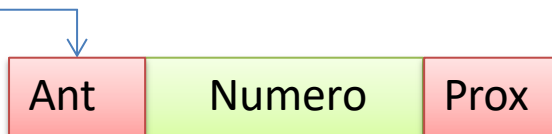
```
{
```

```
TNo *Ant;
```

```
int Numero;
```

```
TNo *Prox;
```

```
};
```



```
struct TLista
```

```
{
```

```
TNo *Primeiro;
```

```
int Qtde;
```

```
TNo *Ultimo;
```

```
};
```



Lista Duplamente Encadeada

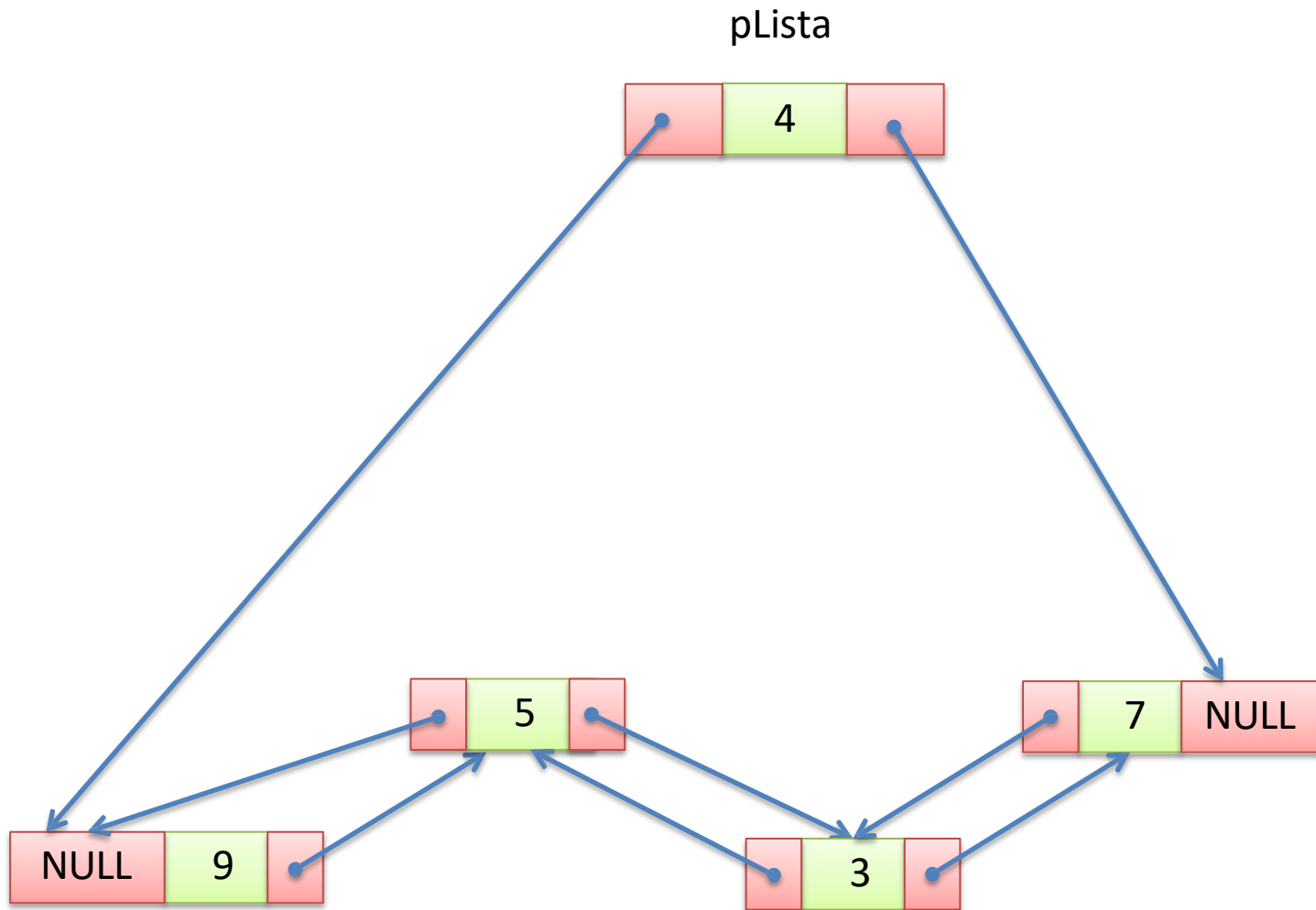


Tabela de Comparações de Listas

Estrutura	Final aponta para	Percorre para trás?
Lista simples	NULL	Não
Lista circular simples	Primeiro nó	Não
Lista dupla	NULL	Sim
Lista circular dupla	Primeiro/último	Sim

A lista circular elimina o fim fixo da lista. A lista dupla permite navegação nos dois sentidos. A lista circular dupla combina essas duas ideias.